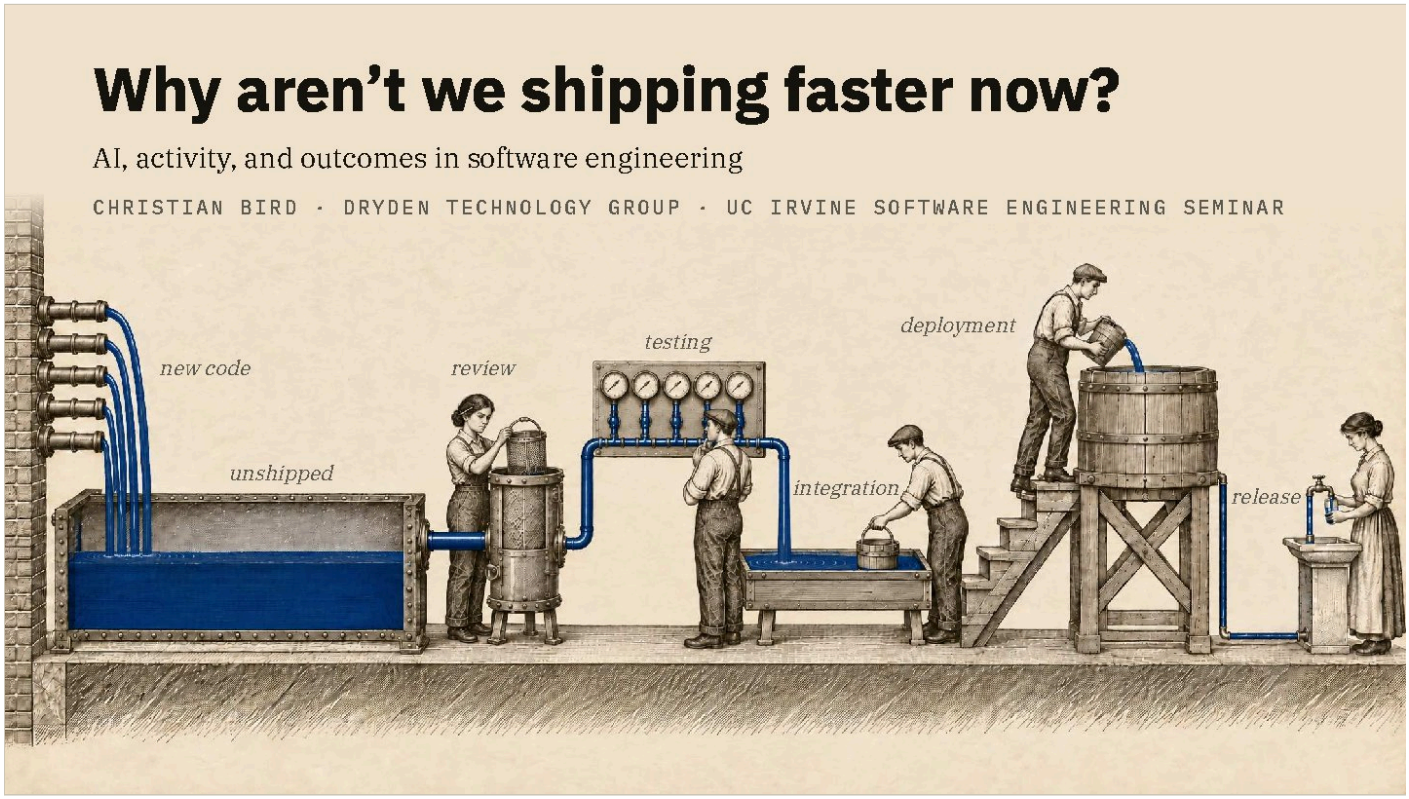
Why aren’t we shipping faster now?

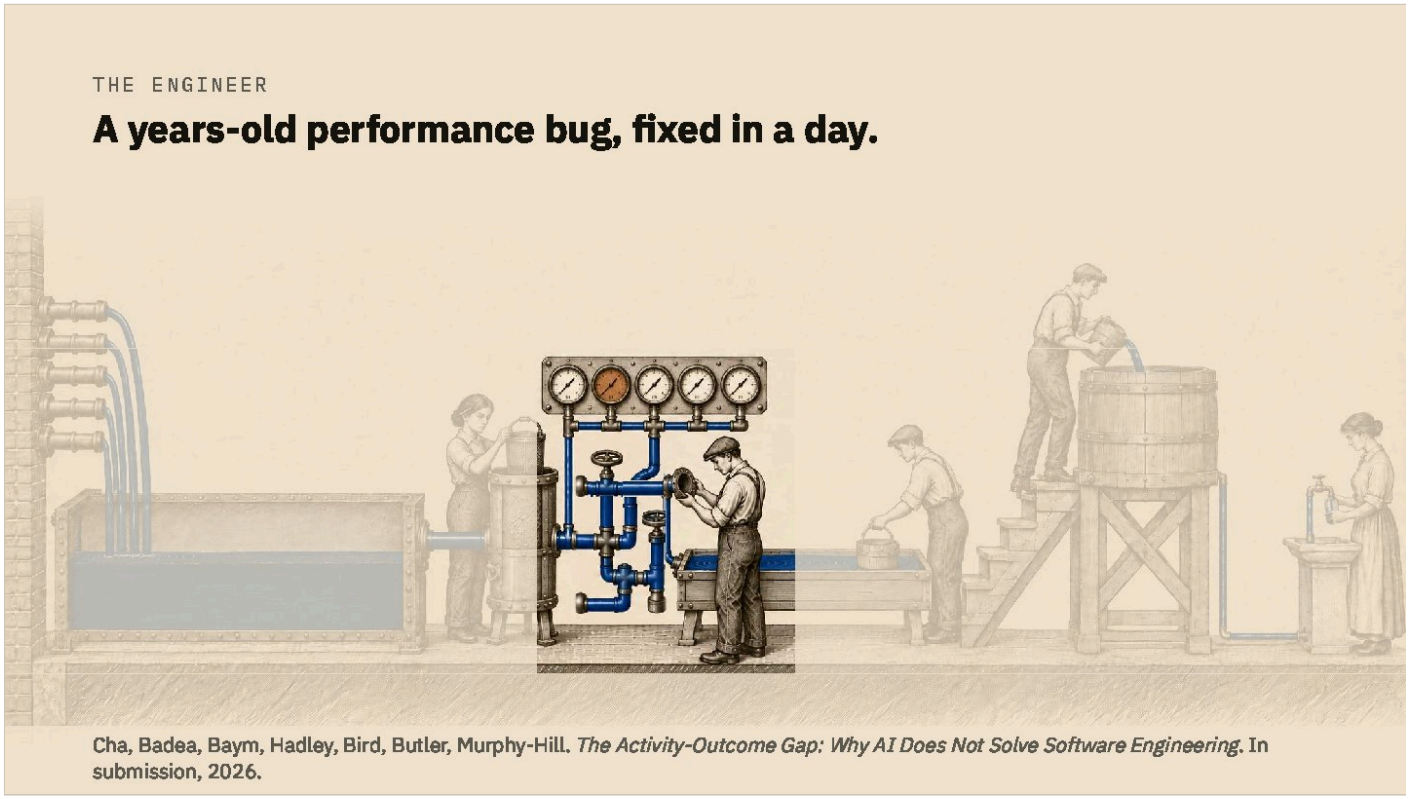


In these pictures the water is code. It comes in on the left as new code, and it is only worth something when it reaches the glass on the right.

The deck with the script underneath: one page per click. 41 pages · 1,869 spoken words · 13.3 minutes at 140 words a minute.

1a Two bug fixes, and the demo

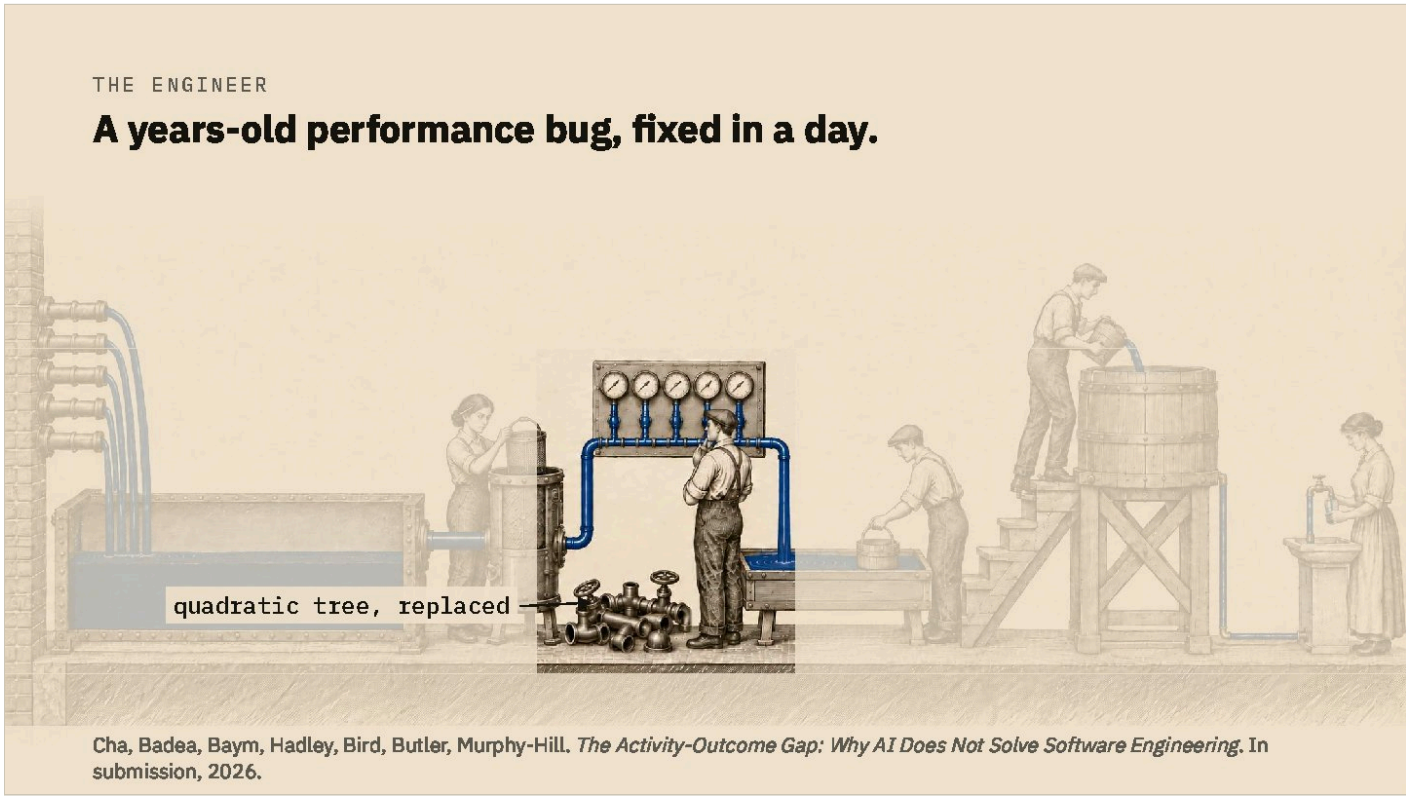
slide at 0:15 · this step ~0:22



A product with millions of users had a performance bug that had been open for years. The last serious attempt to fix it was about a decade ago. This spring an engineer gave a coding agent the context, and the agent found the problem: a tree was being built in quadratic time.

1b Two bug fixes, and the demo

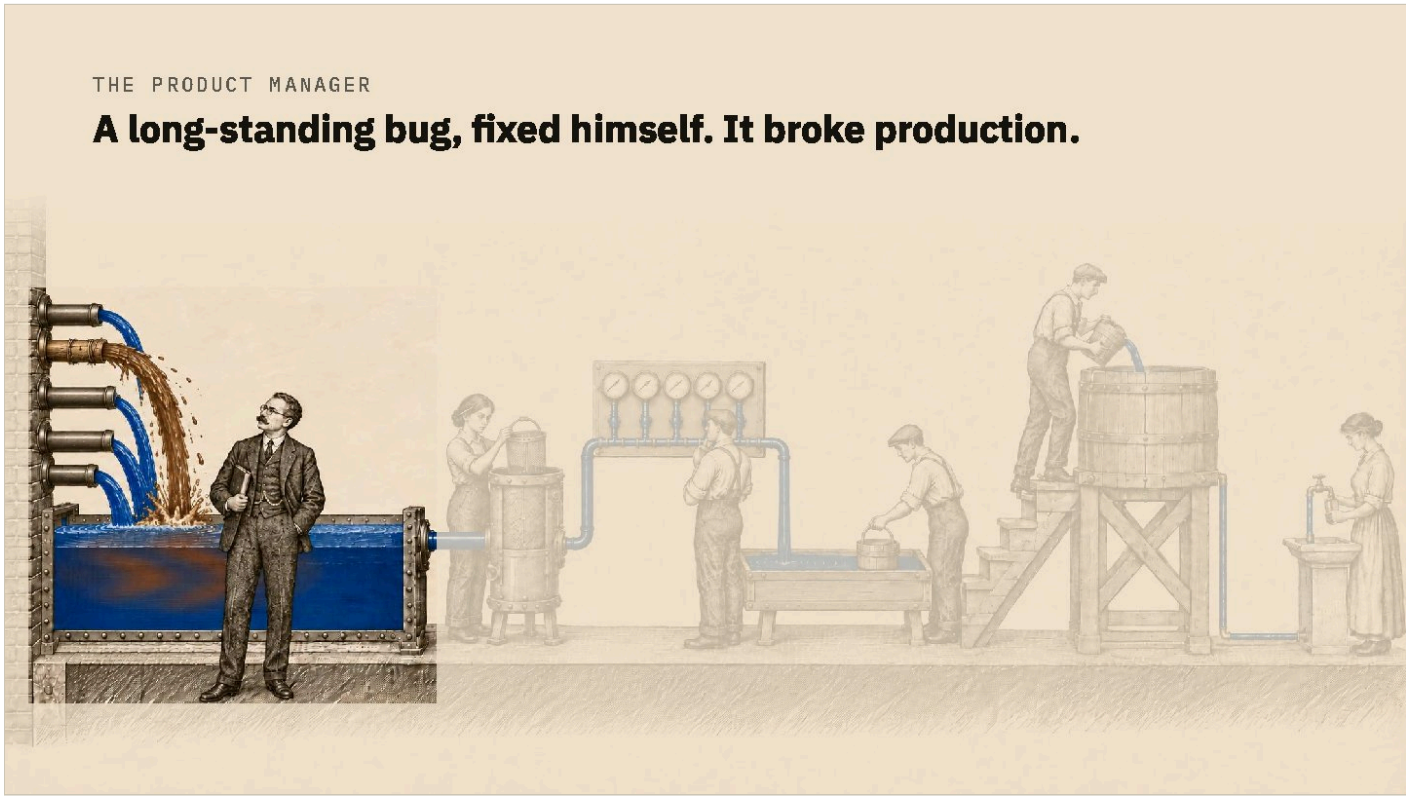
slide at 0:15 · this step ~0:28



He replaced it in a day. He also said something I appreciated: "It was kind of [a] lucky scenario, right? I don't think every problem [is] going to be like this." And he told our intern why he had never tried before. He would have had to justify the work up the management chain first. With the agent he could explore first and show people afterwards.

1c Two bug fixes, and the demo

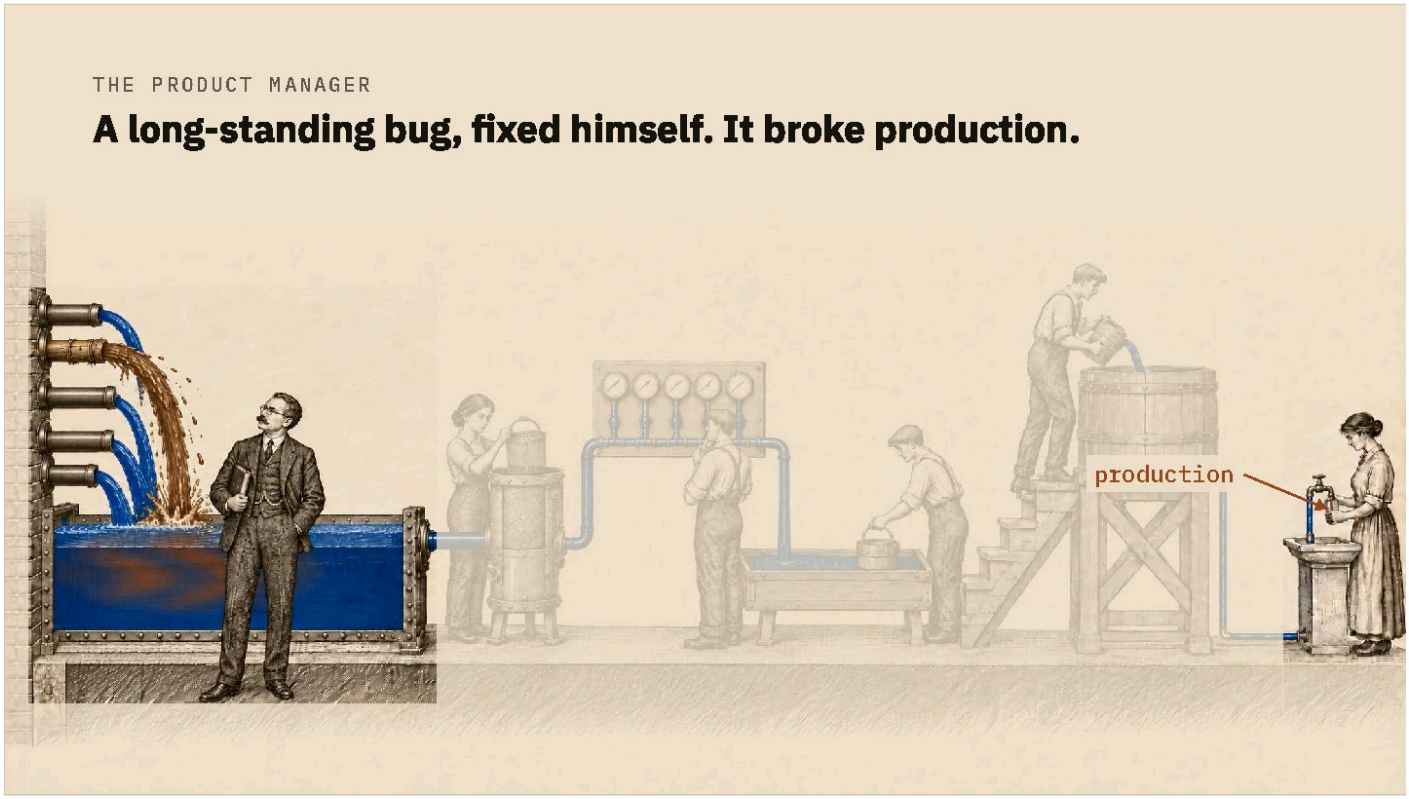
slide at 0:15 · this step ~0:13



A product manager in a coauthor's organization had watched engineers around him do this kind of thing. He picked a long-standing bug, fixed it with the agent, and it broke production.

1d Two bug fixes, and the demo

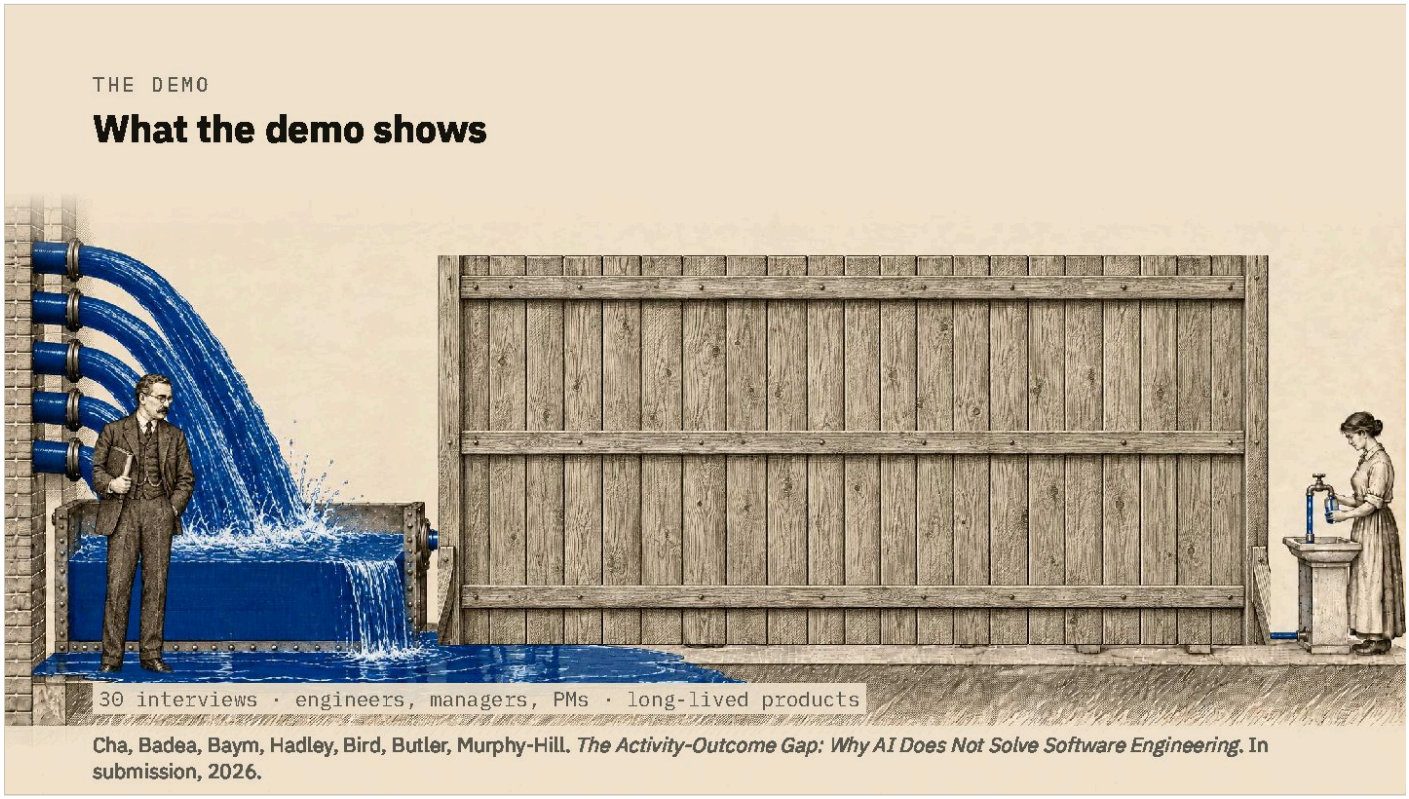
slide at 0:15 · this step ~0:22



In my coauthor's words: "There was something that he didn't realize that the engineers knew, and he broke production." Same tool, same permission. What was different was that the engineer knew what that codebase assumed, and the product manager did not, and neither did the AI. I'll come back to him.

1e Two bug fixes, and the demo

slide at 0:15 · this step ~0:15



The successes are visible and the preparation is not. Another engineer, about a bug his team cracked with an agent: "But how many hours did we spend refining that tool, to crack that bug?"

1f Two bug fixes, and the demo

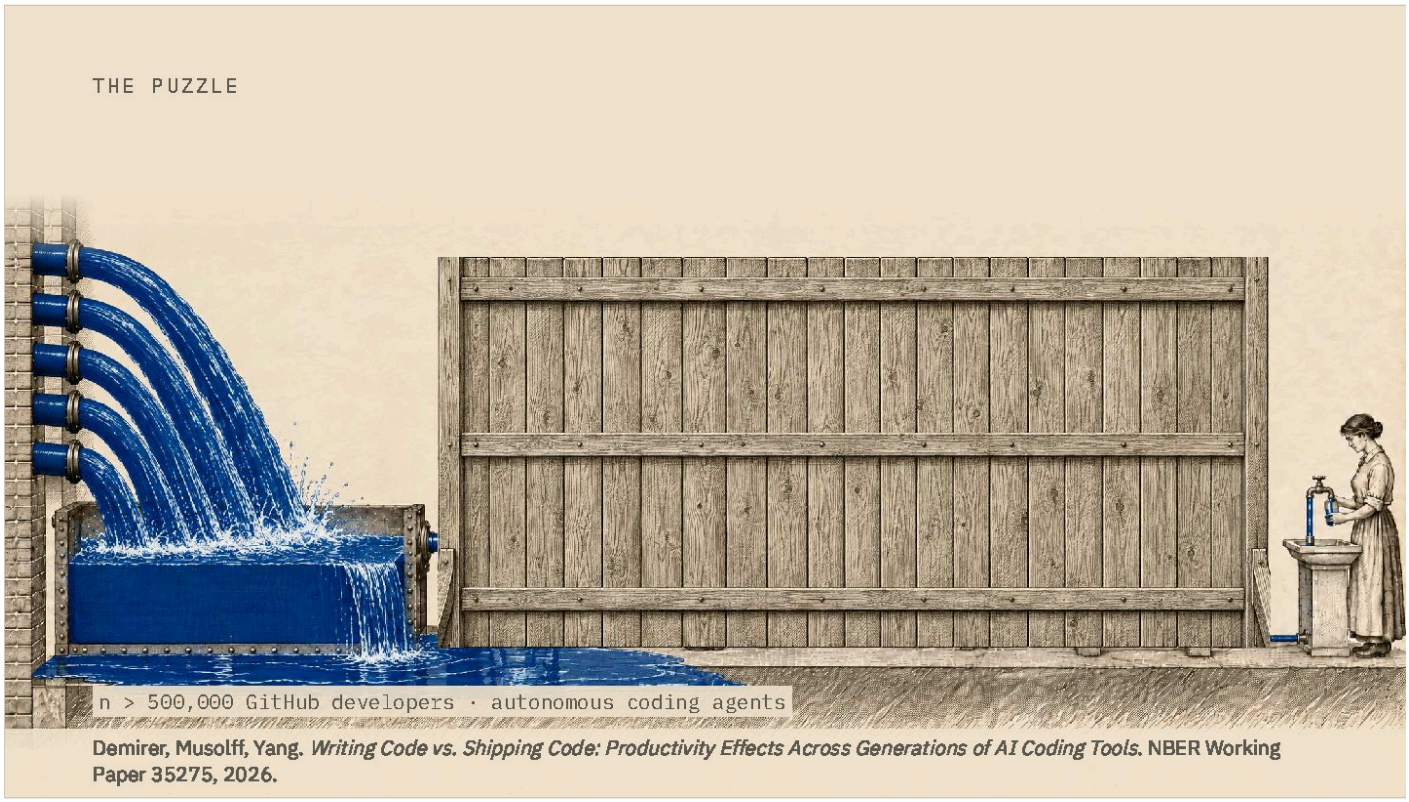
slide at 0:15 · this step ~0:12



Leadership sees the demo and asks the next team, "Why is it so hard for you guys to do the same thing?" That is the question in my title.

2a Activity rose about eight times as much as releases

slide at 2:15 · this step ~0:11

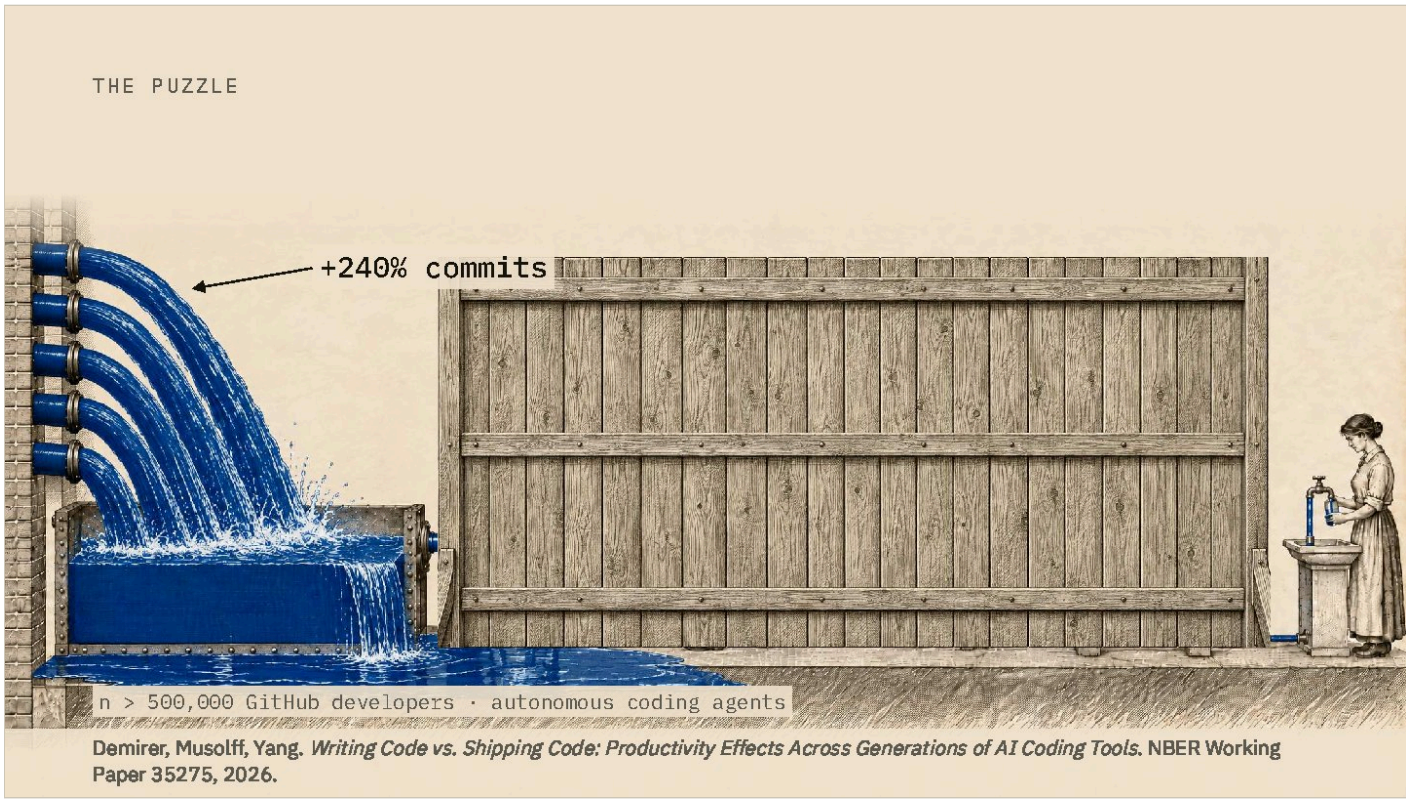


Demirer, Musolff, Yang. *Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools.* NBER Working Paper 35275, 2026.

Here is what it looks like at scale. A study of more than 500,000 GitHub developers looked at what happened as the tools became more autonomous.

2b Activity rose about eight times as much as releases

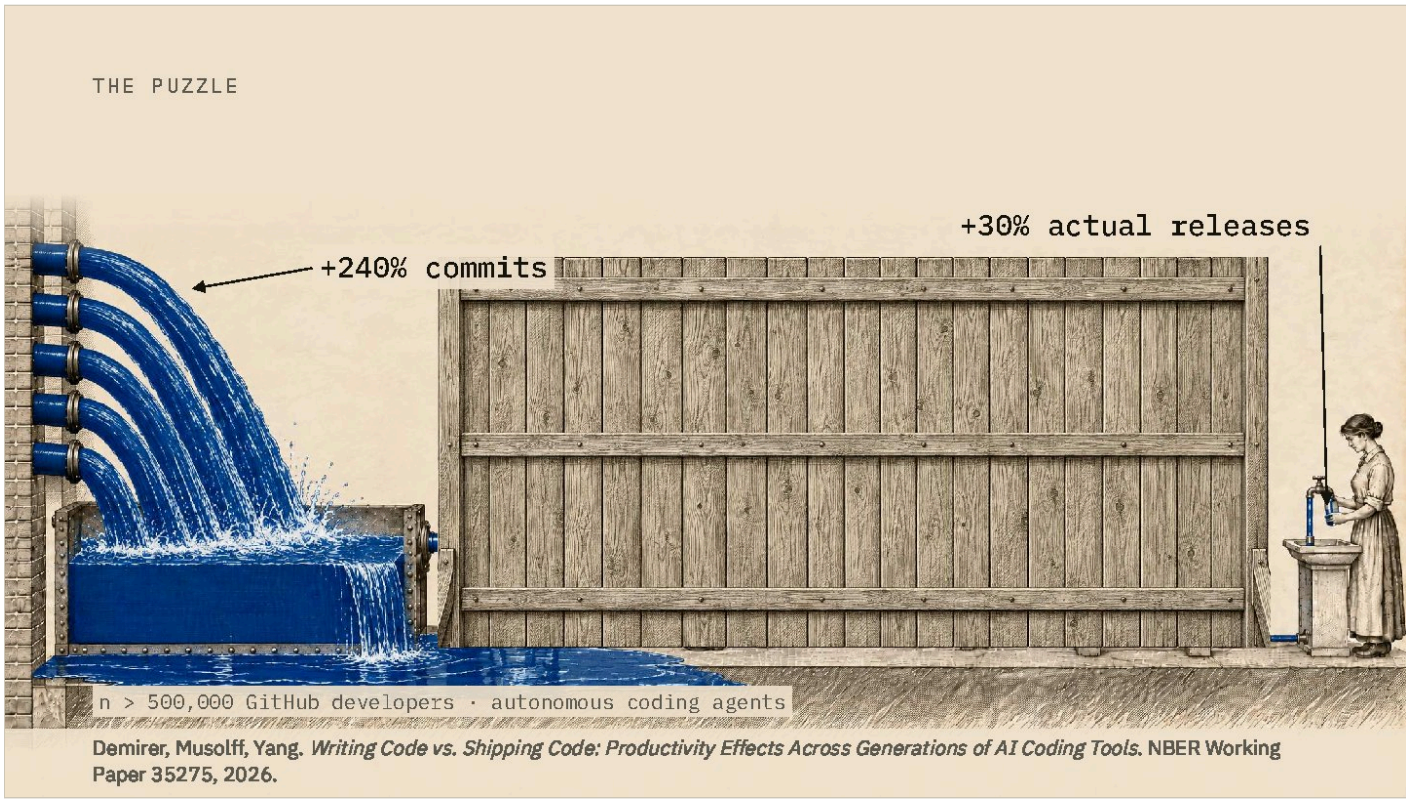
slide at 2:15 · this step ~0:05



For developers using autonomous coding agents, commits went up by 240 percent.

2c Activity rose about eight times as much as releases

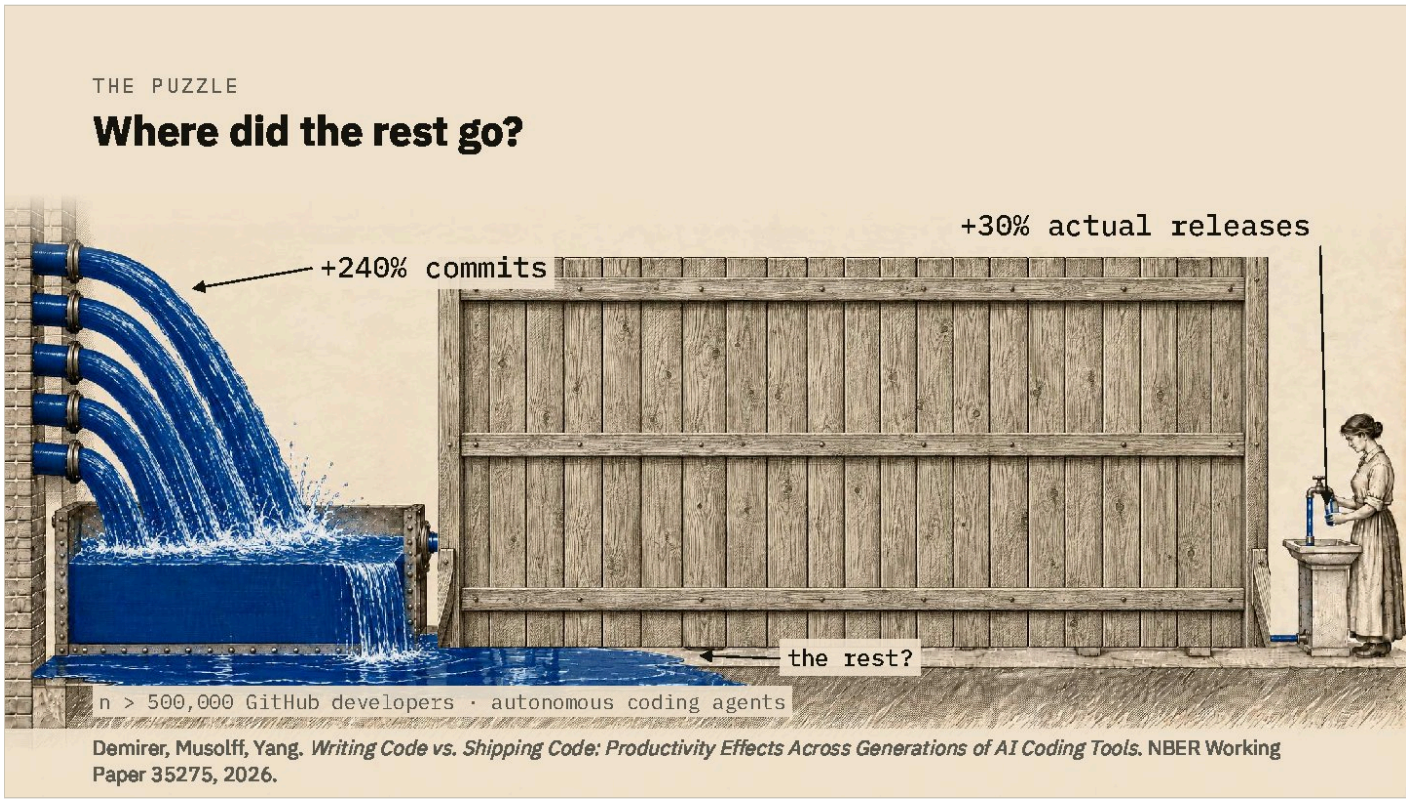
slide at 2:15 · this step ~0:12



Actual releases went up 30 percent. The authors call it the weak-link hypothesis: the gains from AI are held back by human bottlenecks in the production chain.

2d Activity rose about eight times as much as releases

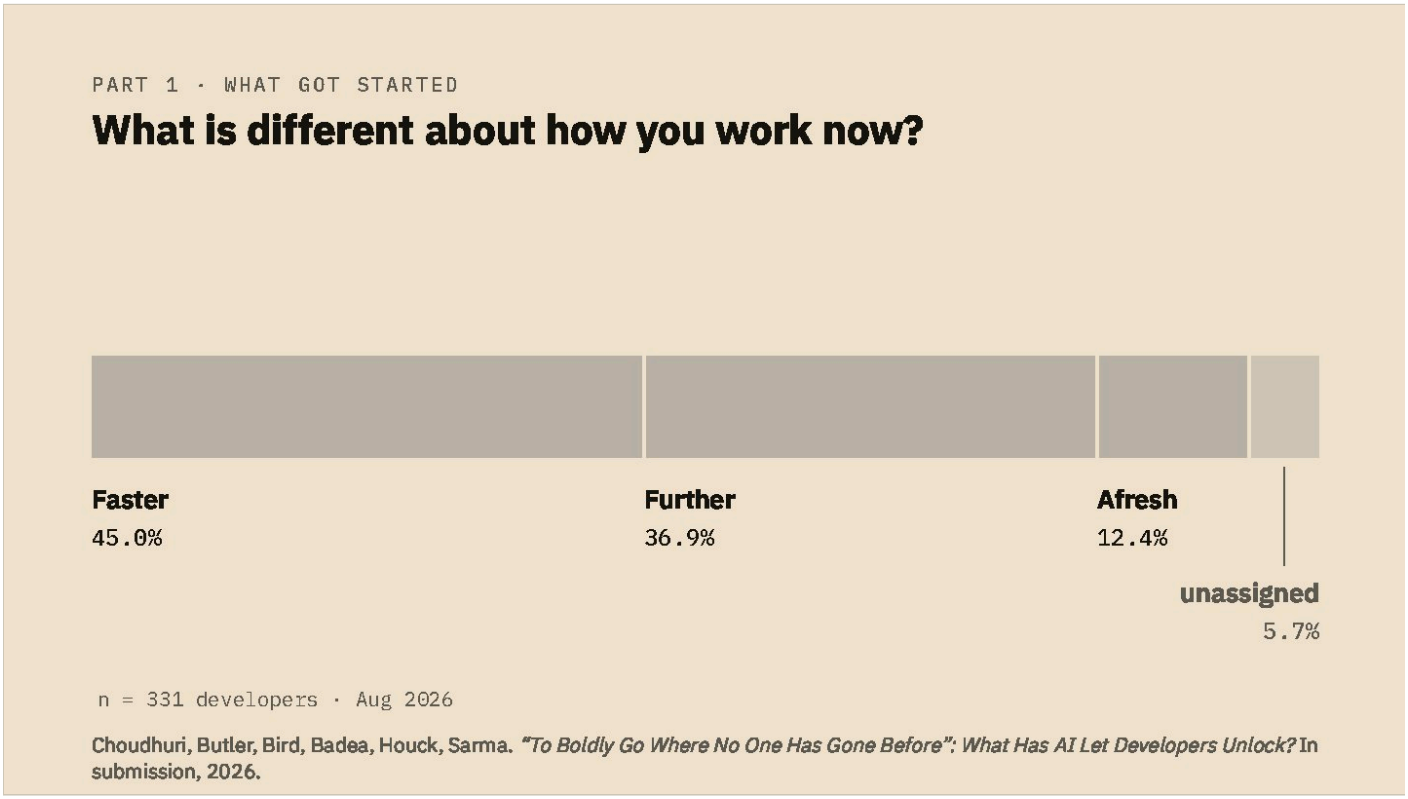
slide at 2:15 · this step ~0:33



Where did the rest go? Behind the fence. I spent sixteen years at Microsoft Research studying how developers work, I moved to Dryden in September, and I am still studying them. For the last two years my coauthors and I have been trying to answer that question, and I'll give you the answer in three parts: what got started, what did not keep up, and why better models do not close the gap on their own.

3a Part 1. What AI let people start

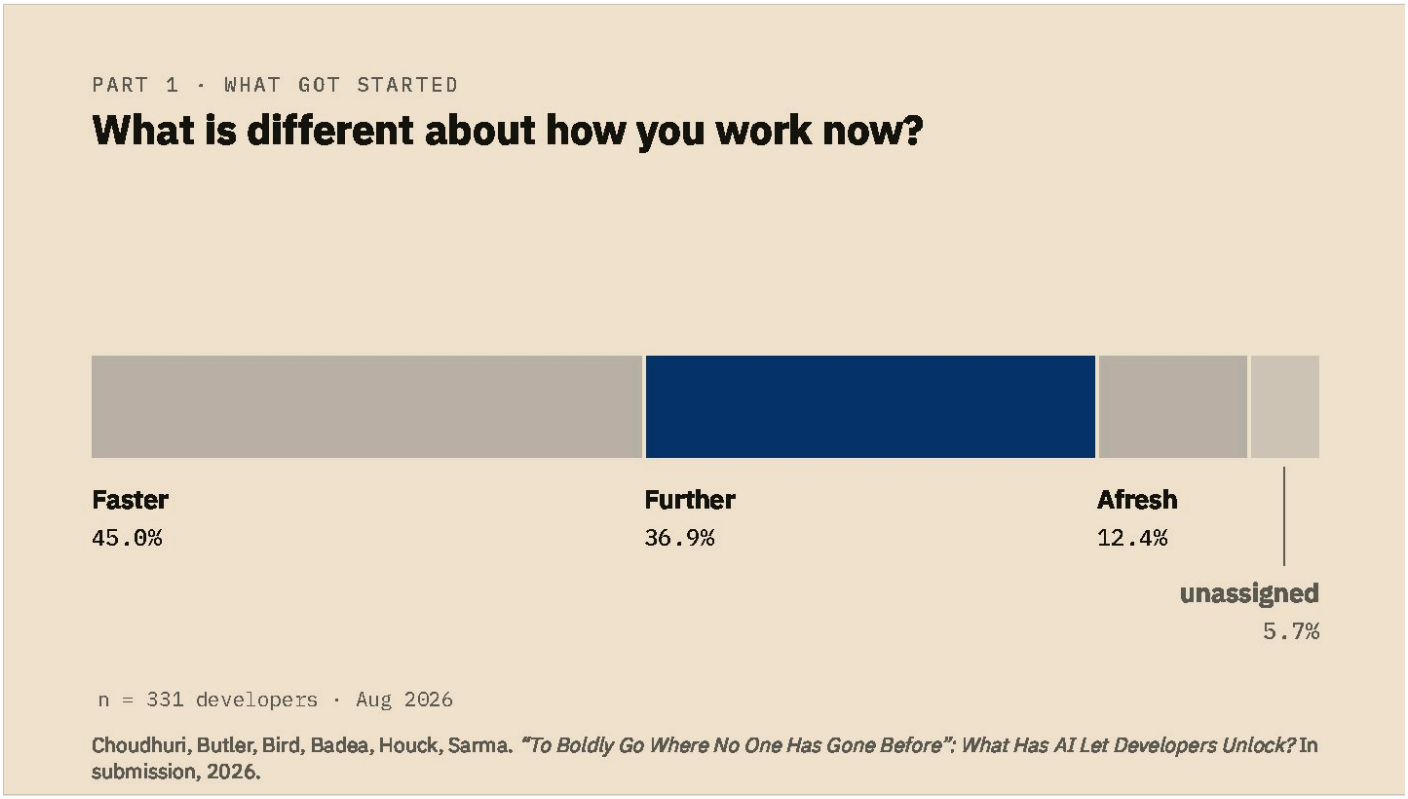
slide at 3:15 · this step ~0:16



Most studies ask how much faster developers do the work they already did. We asked 331 developers what they can do now that they could not do before. Forty-five percent of the answers were the old work, faster.

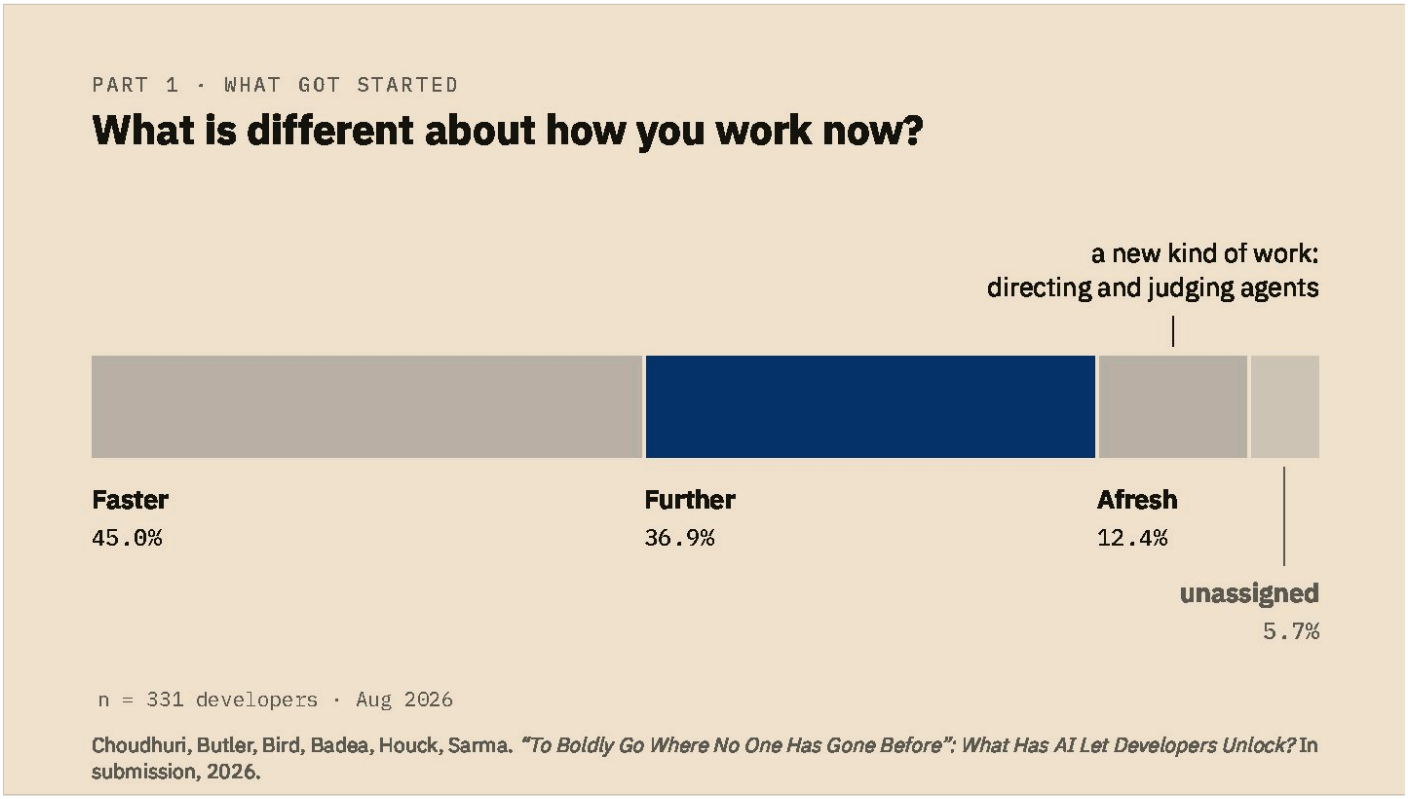
3b Part 1. What AI let people start

slide at 3:15 · this step ~0:24



Thirty-seven percent were work that had been out of reach, things people had wanted to do and could never justify. One of them: "I recently automated it in an hour that saved me 80 hours of manual work. I never would have done those 80 hours." That thirty-seven percent is new water entering the pipe.

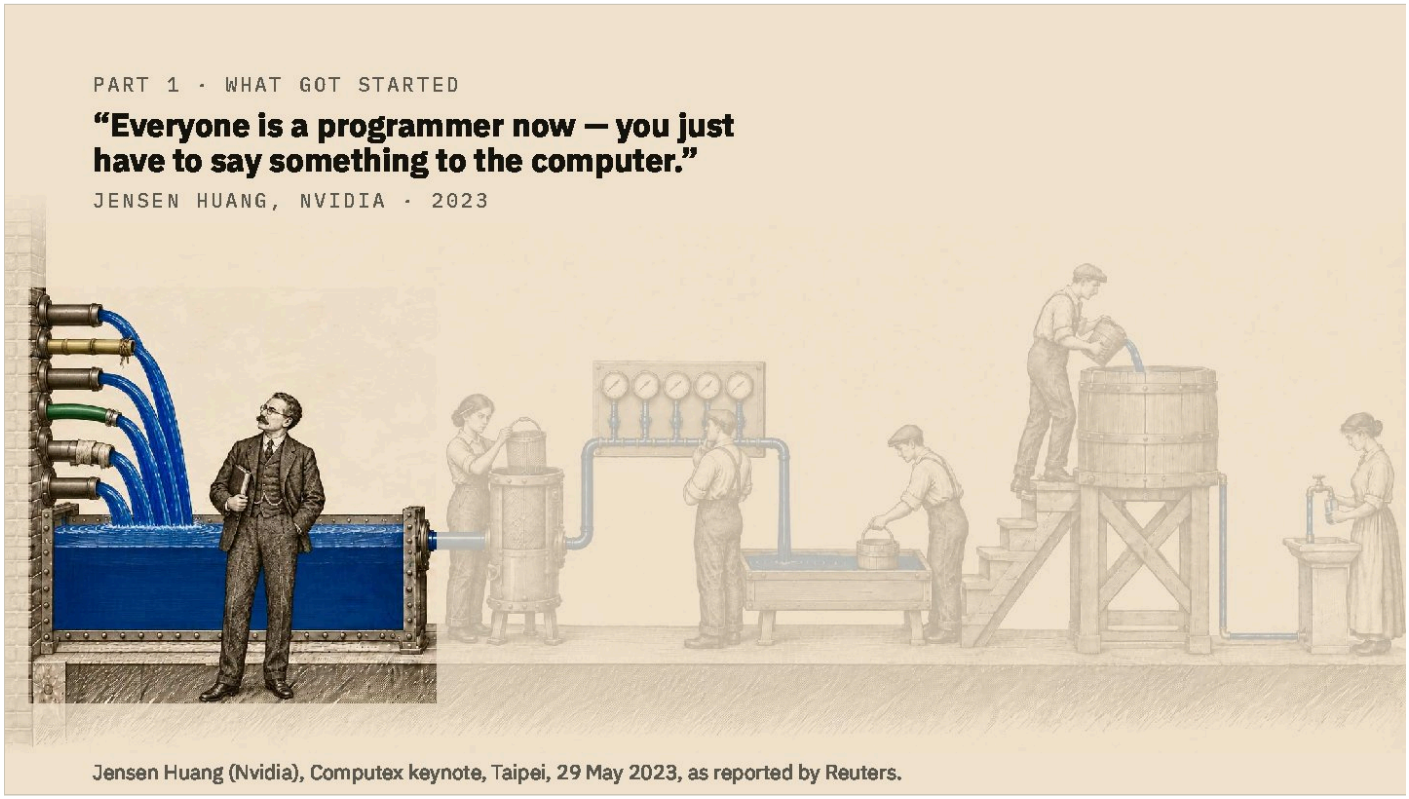
3c Part 1. What AI let people start



Another twelve percent is different, and it matters for the rest of the talk. It is a new kind of work that grew up around the AI: "directing AI, deciding when its work was done, and building what it needs." That is supervision, and we will see it again.

4a More hands, less context

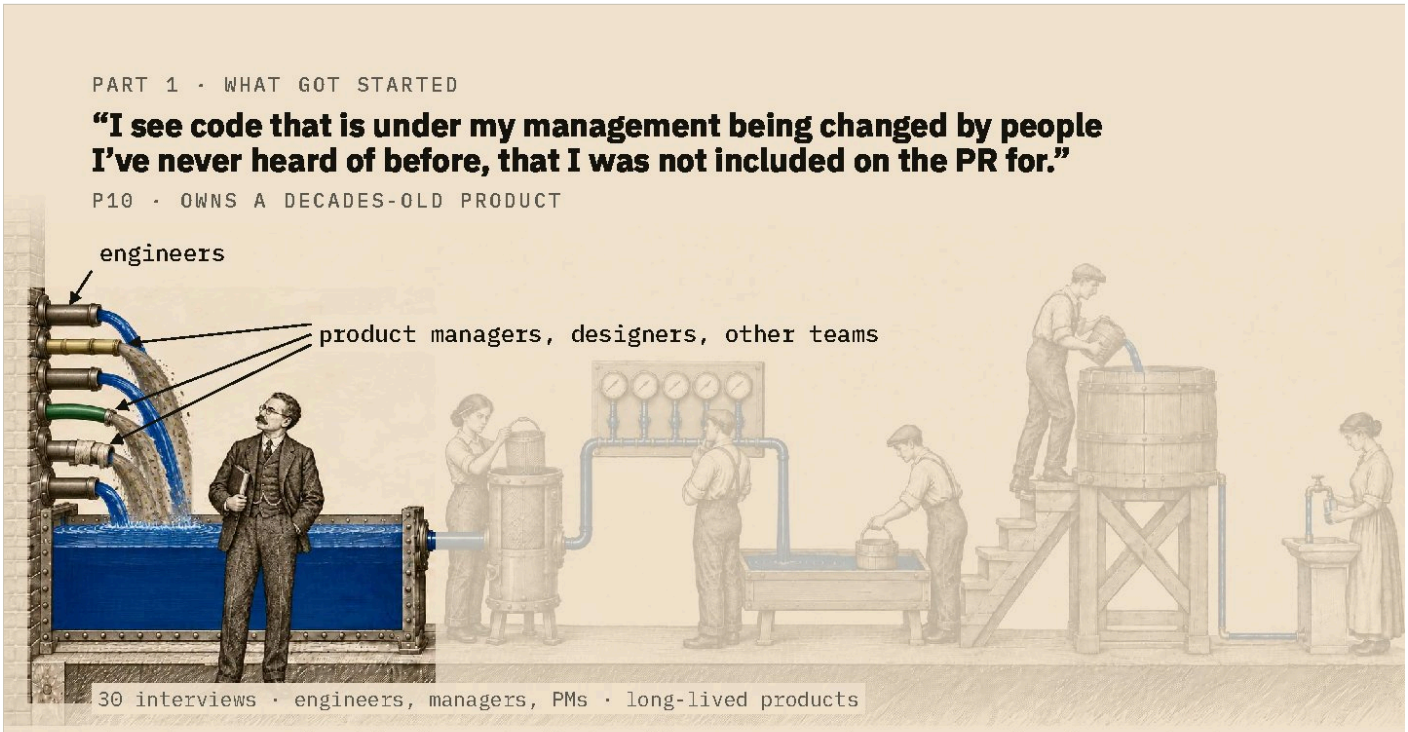
slide at 4:15 · this step ~0:09



In 2023 Nvidia's CEO said, "Everyone is a programmer now — you just have to say something to the computer."

4b More hands, less context

slide at 4:15 · this step ~0:48



The new water does not come only from developers. In Yoonha Cha's thirty interviews with engineers, managers and product managers on long-lived products, one finding was that "AI expanded who could initiate software changes." A senior engineer described his code "being changed by people I've never heard of before, that I was not included on the PR for." And what the new contributors do not bring is what the codebase assumes. One participant submitted an AI-written fix and got back the comment "You completely broke an accessibility design contract." No bot had caught it. So: more work, from more hands, carrying less of the context. Here is what happens to that work next.

5a The moment: one day to code it, thirty days to ship it

slide at 5:15 · this step ~0:15



One engineer said this, and I think it is the most useful sentence in the study: "If you can do the code in one day, you're still talking 30 days before it actually gets deployed."

5b The moment: one day to code it, thirty days to ship it

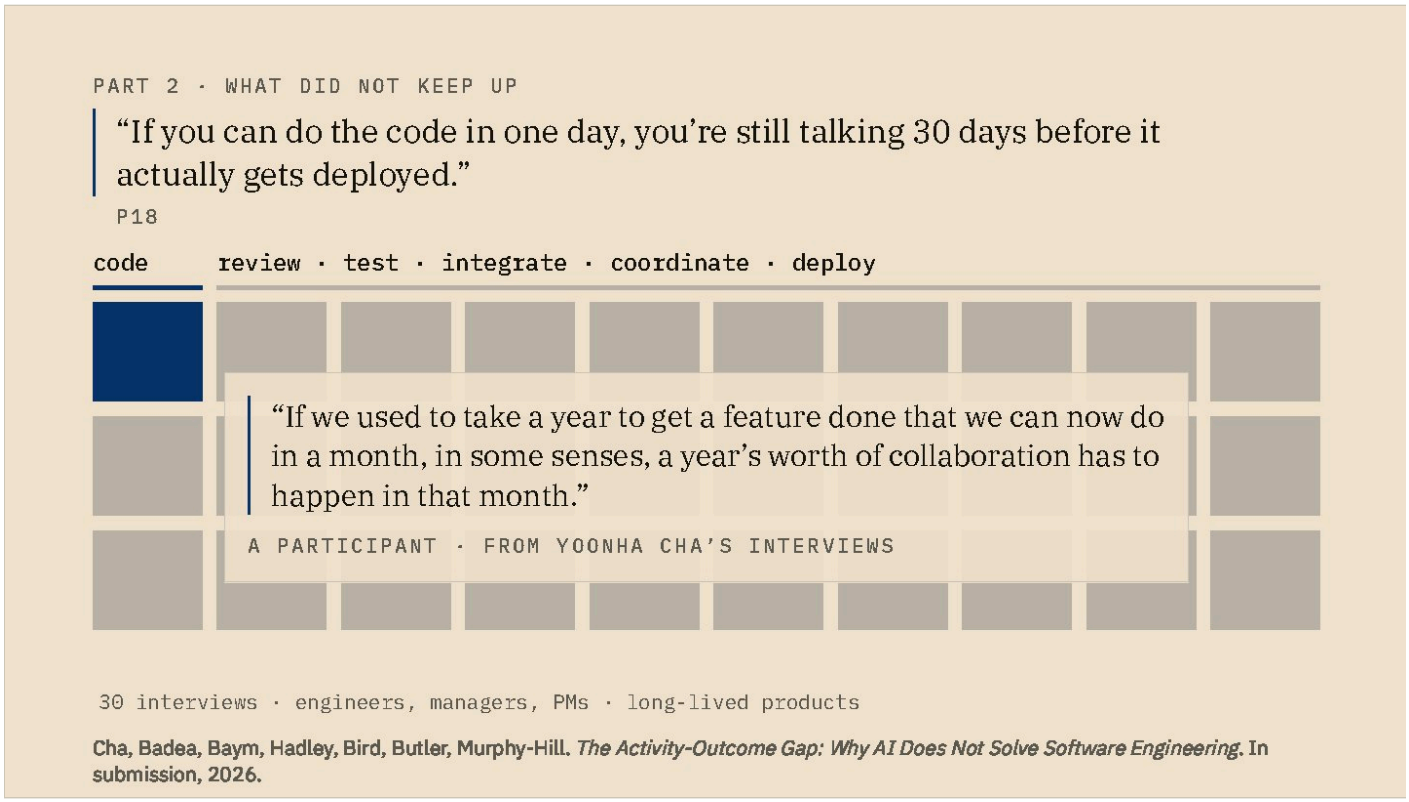
slide at 5:15 · this step ~0:21



Another, on a new feature for a product that is six million lines of code written over forty years, said the hard part was never the feature: "It was integrating it with the existing 6 million lines of code written over 40 years. And the agent also sucks at it."

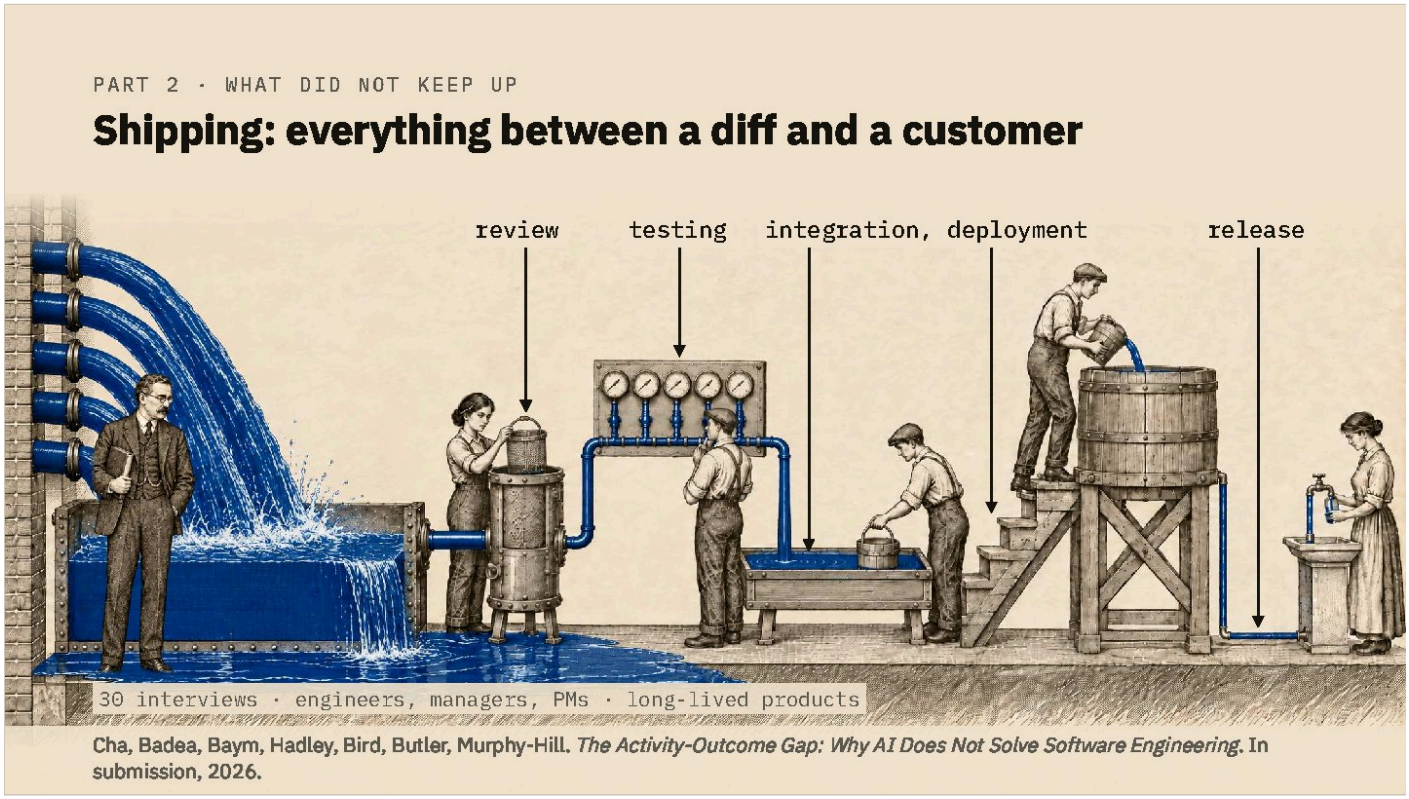
5c The moment: one day to code it, thirty days to ship it

slide at 5:15 · this step ~0:18



A participant put the general version well: "If we used to take a year to get a feature done that we can now do in a month, in some senses, a year's worth of collaboration has to happen in that month."

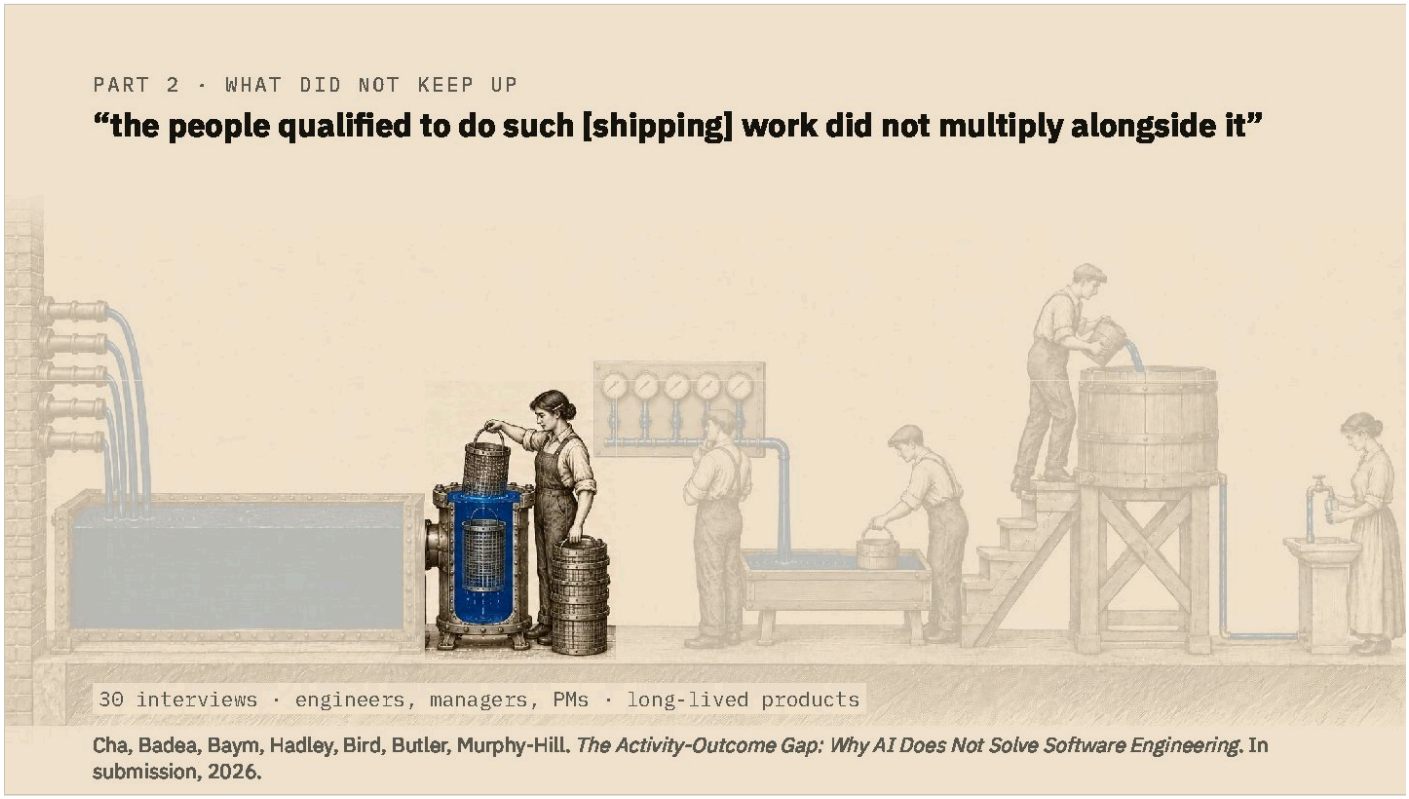
6a The people who can ship it did not multiply



Take the fence away, and this is what the demo was hiding. Call everything between a diff and a customer shipping: review, testing, integration, deployment, release, and the coordination between them.

6b The people who can ship it did not multiply

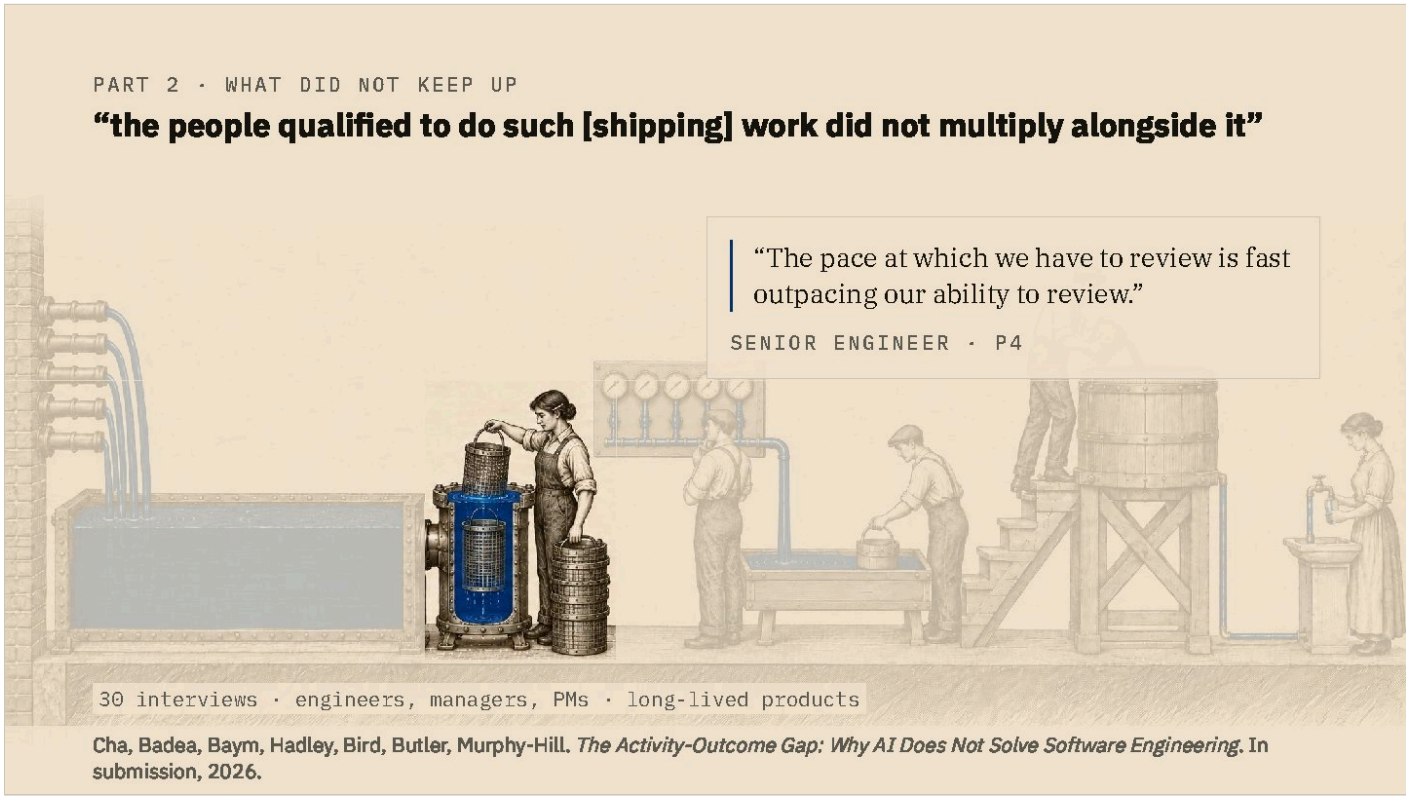
slide at 6:15 · this step ~0:12



Yoonha's paper calls it conversion work, and its finding is that the people qualified to do it "did not multiply alongside it." Let me make that one person.

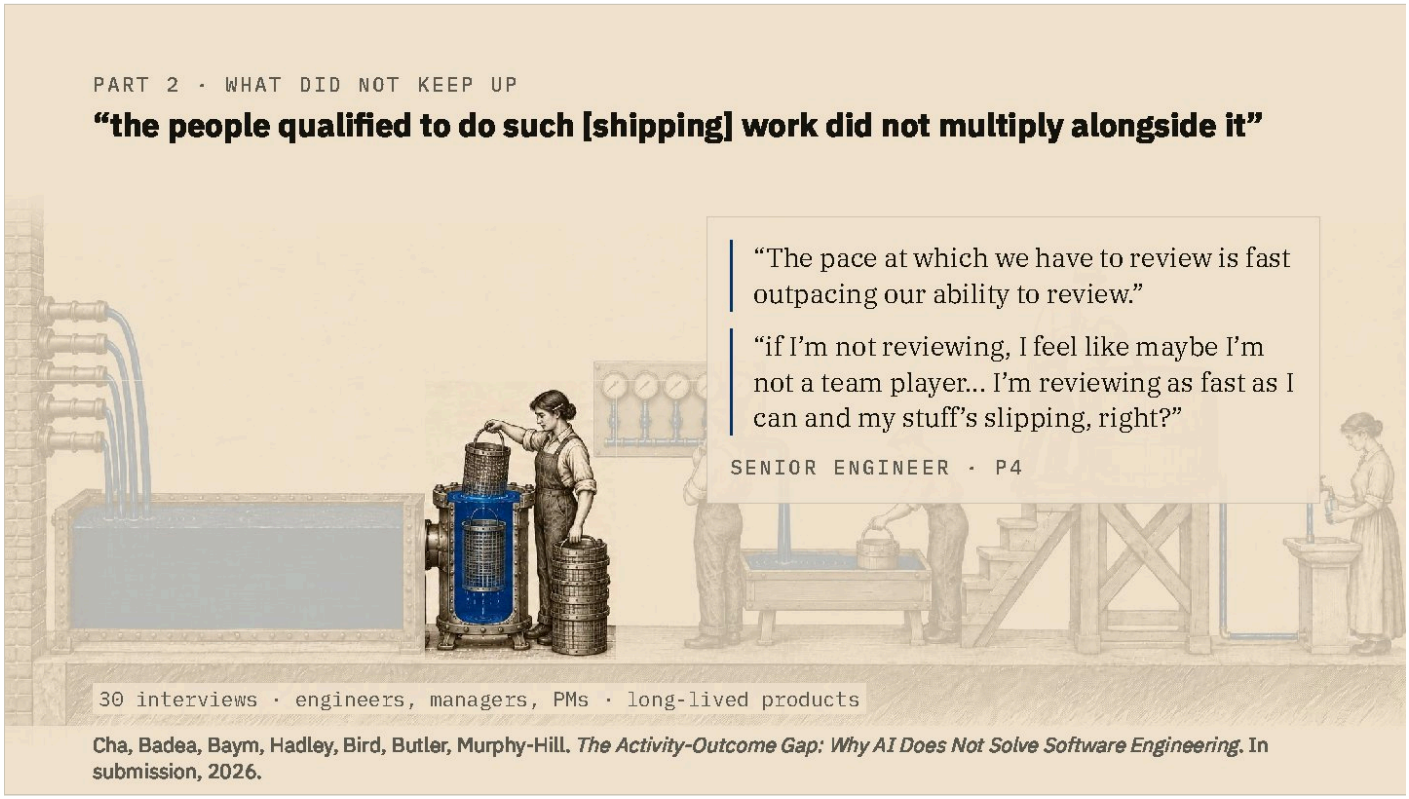
6c The people who can ship it did not multiply

slide at 6:15 · this step ~0:12



A senior engineer told Yoonha that "The pace at which we have to review is fast outpacing our ability to review." And they keep reviewing, because, in their words,

6d The people who can ship it did not multiply



"if I'm not reviewing, I feel like maybe I'm not a team player... I'm reviewing as fast as I can and my stuff's slipping, right?" I suspect that is what happened to the product manager's change: the people best equipped to look it over were the least available to do it.

7a Expectations rose, and work got dropped

slide at 7:15 · this step ~0:35



They were the least available because expectations rose faster than capacity, set by demos like the one at the start. The new expectations were mostly never stated. They "appeared indirectly through heavier workloads and shorter timelines." So work gets dropped. Here is what gets dropped. "When everything is P0, everything is more important, and everything is a fire, certain guardrails, processes, and things fall through the cracks." One change nearly exposed an API without security approval and had to be reverted.

7b Expectations rose, and work got dropped

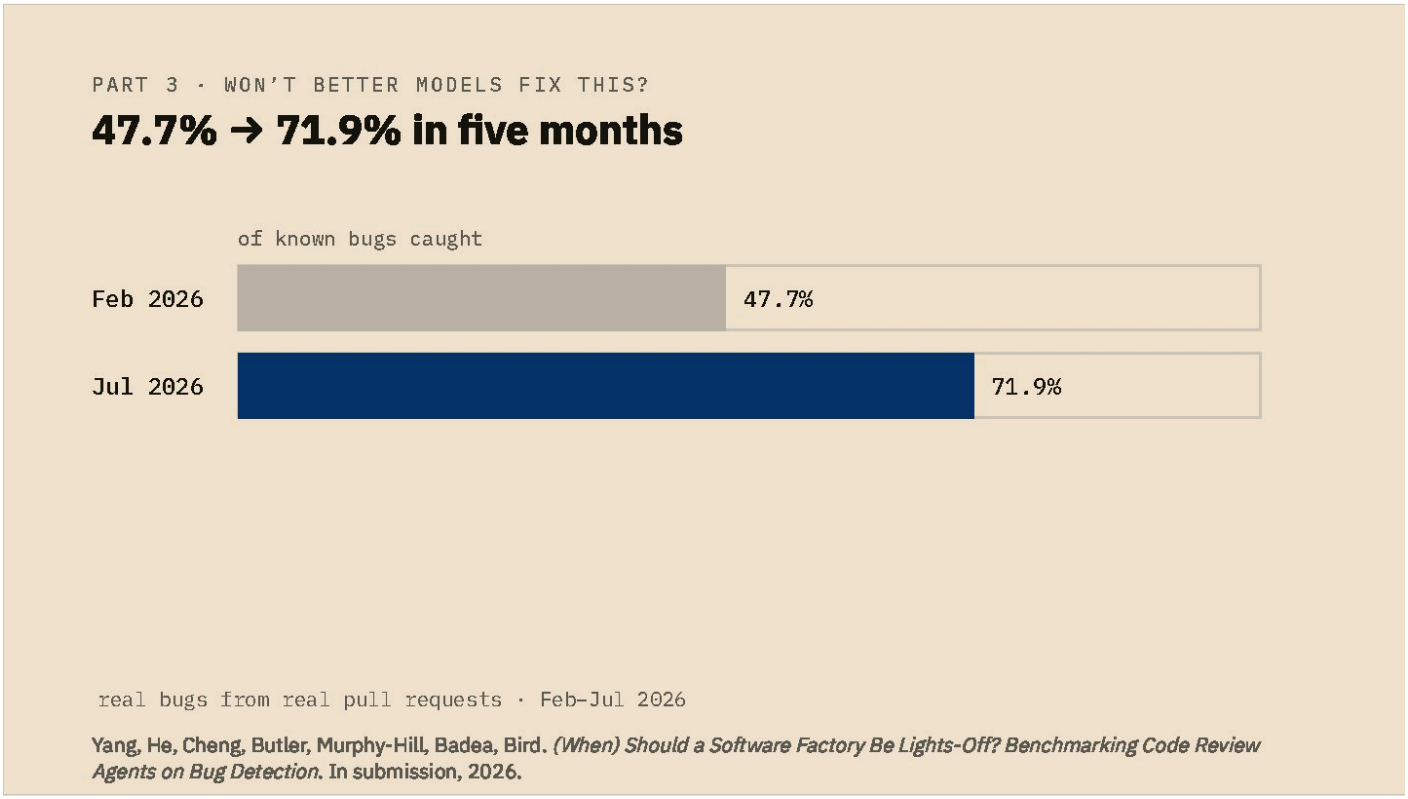
slide at 7:15 · this step ~0:21



The paper calls this pattern Systemic Conversion Strain: the cost of generating code fell, while the cost of shipping it barely moved, and expectations treated the two as the same. So that is where the extra activity went: into new work, into more hands, into review queues, and into reverts.

8a Part 3. Won't better models fix this?

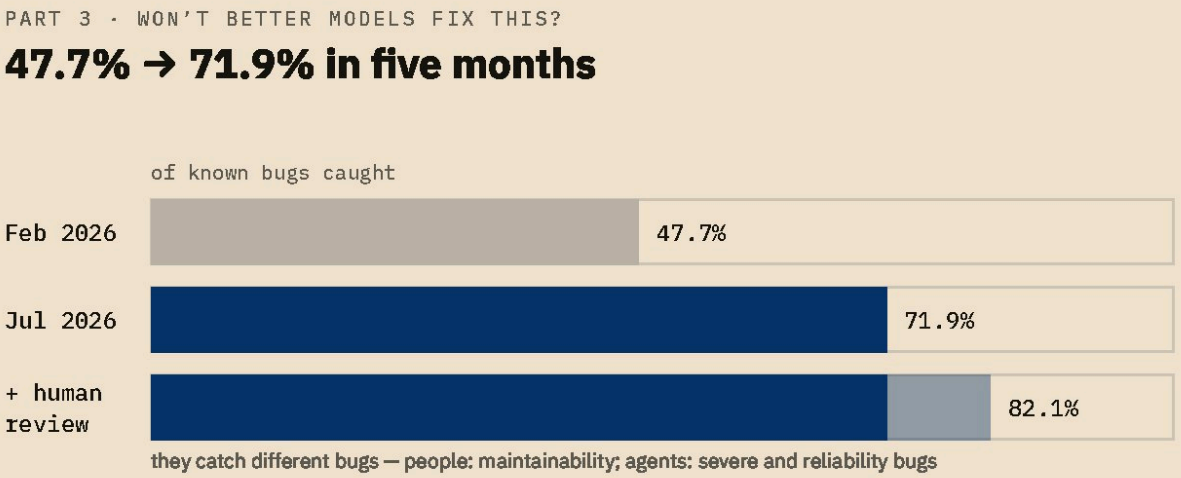
slide at 8:15 · this step ~0:18



Won't better models fix this? They are getting better. On a benchmark built from real bugs in real pull requests, AI reviewers went from catching 47.7 percent of them to 71.9 percent in five months, from one model generation to the next.

8b Part 3. Won't better models fix this?

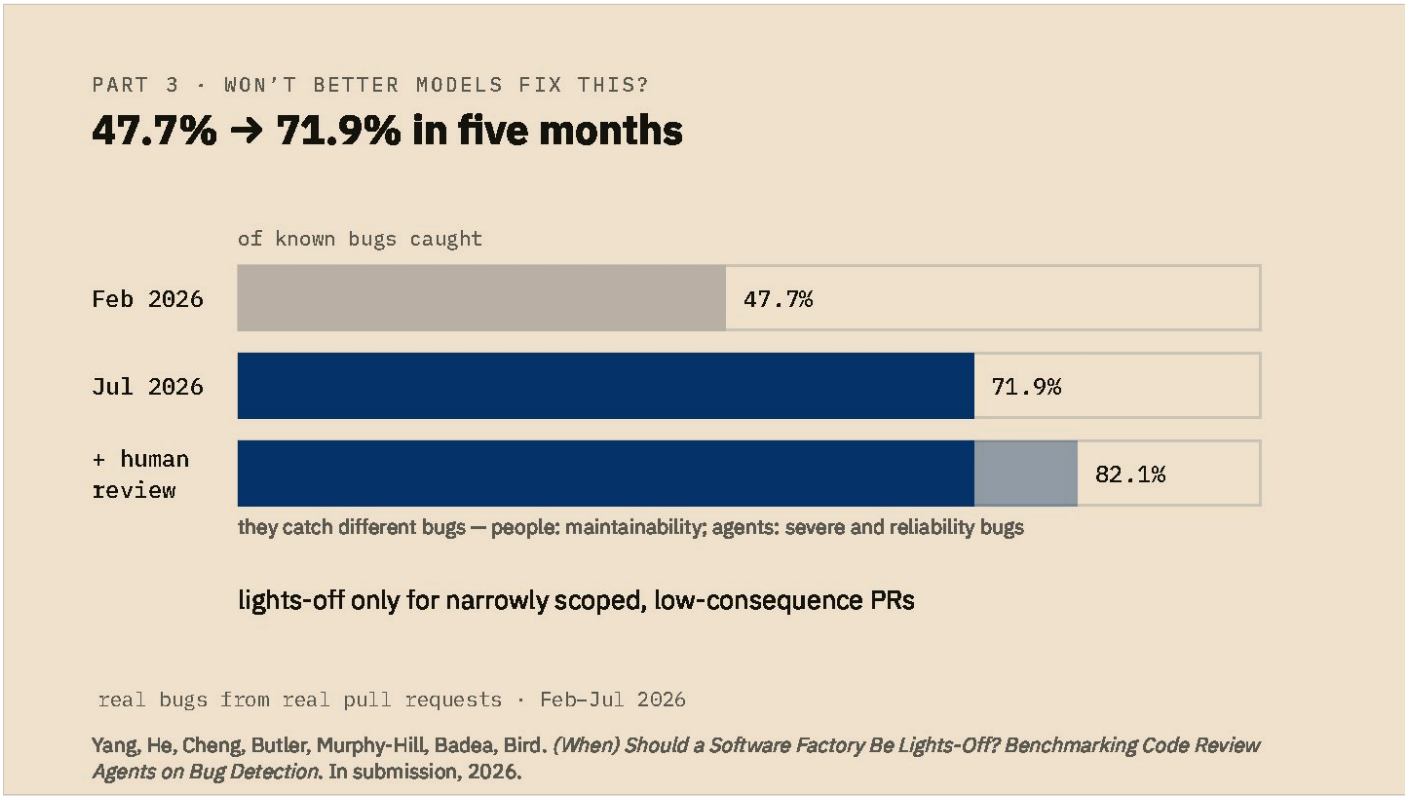
slide at 8:15 · this step ~0:18



Put the best agent together with the human review those pull requests actually got, and together they caught 82.1 percent, because they catch different bugs: people catch more of the maintainability problems, and agents more of the severe and reliability bugs.

8c Part 3. Won't better models fix this?

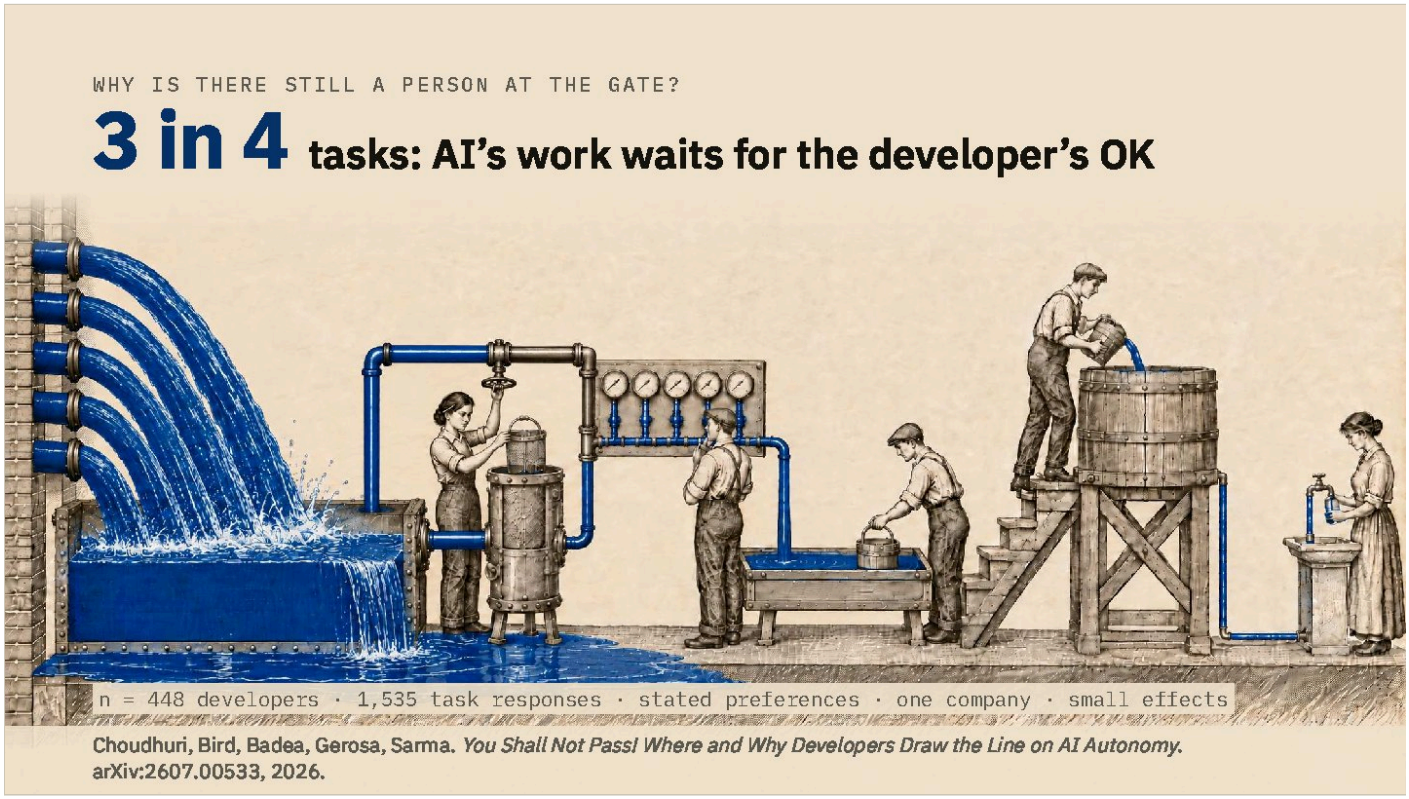
slide at 8:15 · this step ~0:09



The authors' conclusion is that review with no human in it holds up today only for "narrowly scoped, low-consequence PRs."

9a Why there is still a person at that gate

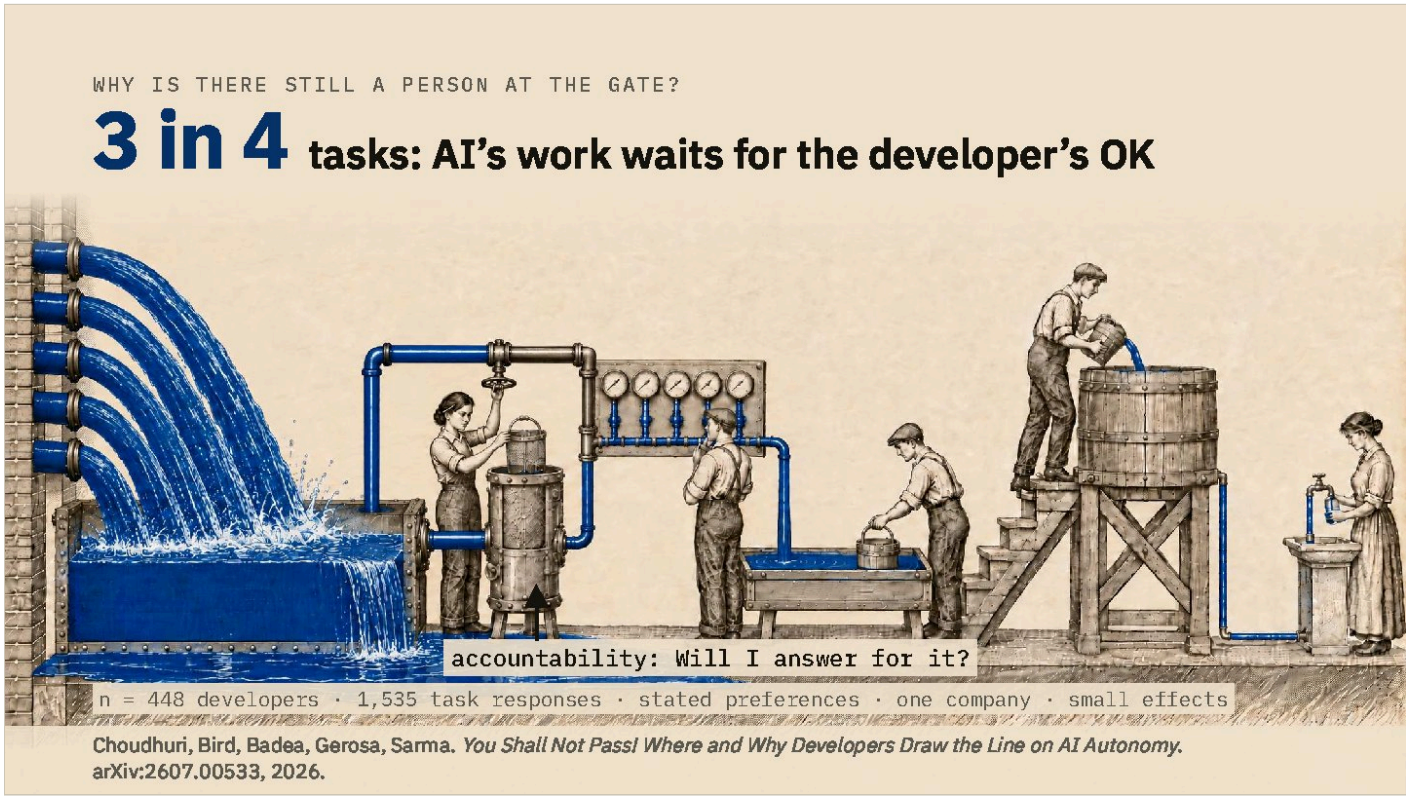
slide at 9:00 · this step ~0:33



So why is there still a person at that gate? We asked 448 developers at one large company, task by task, how much control they would give an AI. Three in four answers would not let the AI's work take effect without a developer signing off. I had a bet about what explains where people draw that line. On tape, before the results, I said accountability: "I would have guessed that would be actually the most important."

9b Why there is still a person at that gate

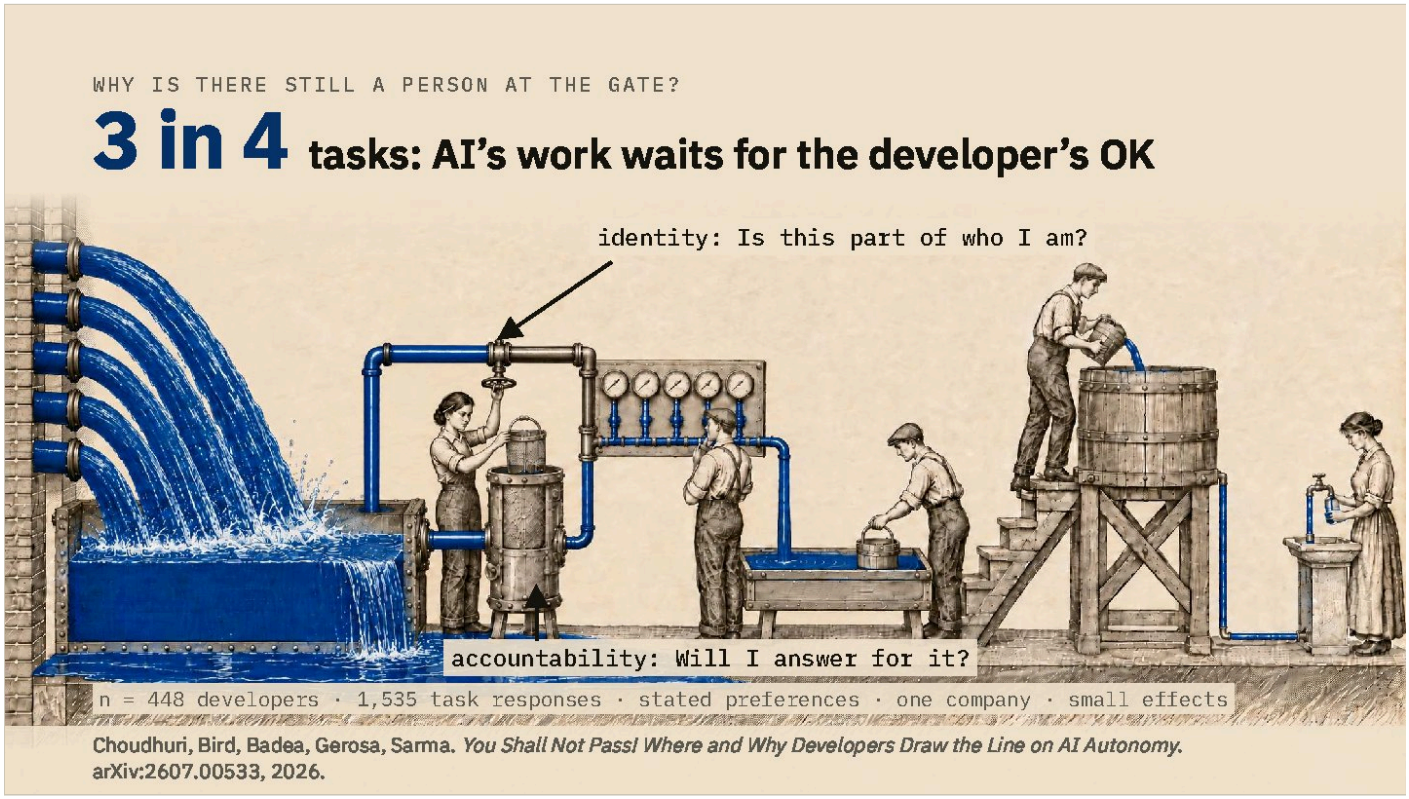
slide at 9:00 · this step ~0:18



Accountability does matter, one step earlier than I thought. It decides whether the AI may do the work at all. One of them, on code review: "marking my approval puts my name on it and makes me partially responsible for it."

9c Why there is still a person at that gate

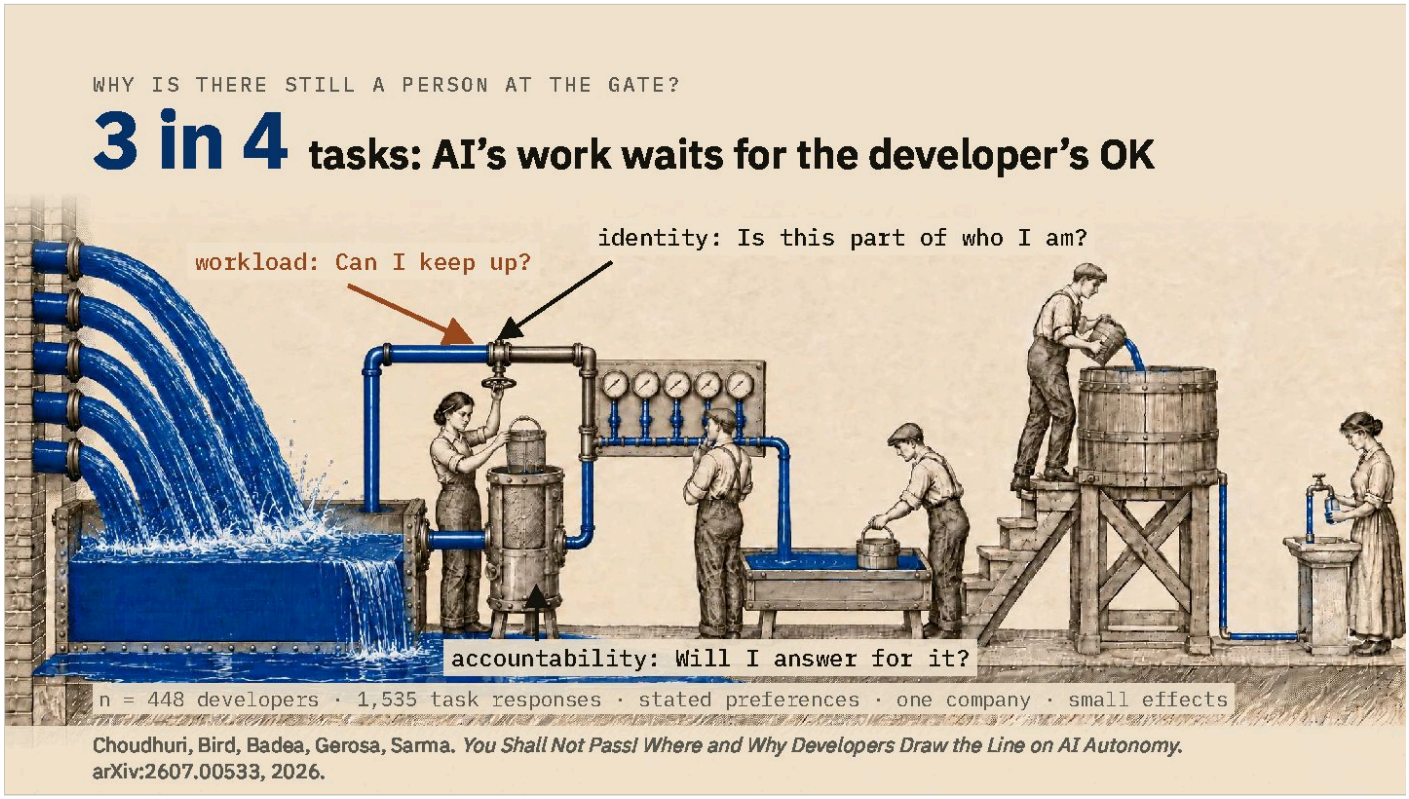
slide at 9:00 · this step ~0:13



What decides the sign-off itself is identity. The more a task is part of who the developer is, the less willing they are to let the AI decide without them.

9d Why there is still a person at that gate

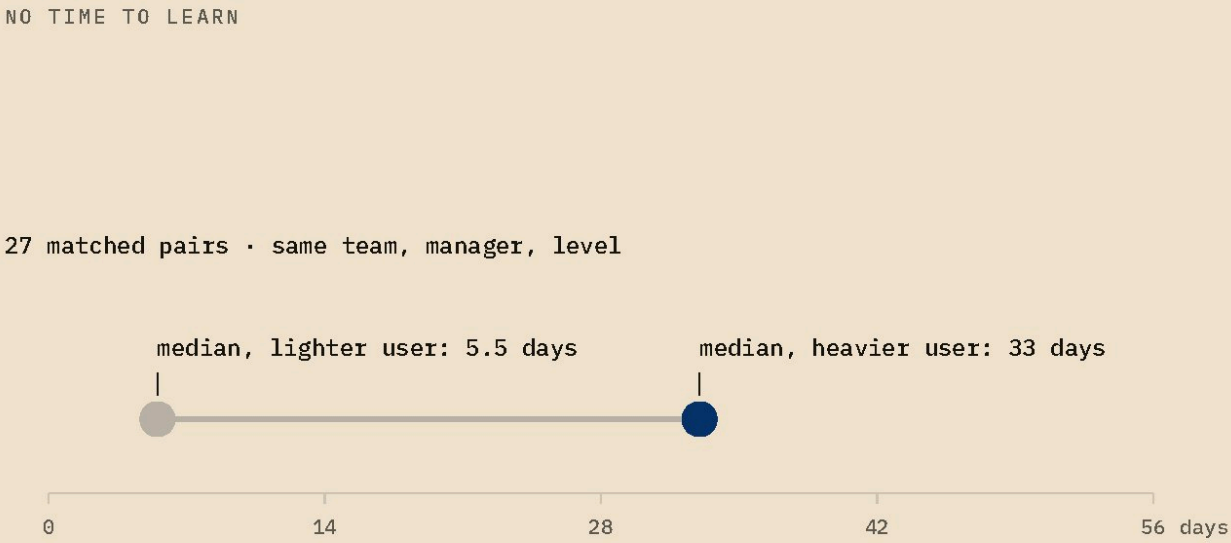
slide at 9:00 · this step ~0:40



And one thing pushes the other way: workload. The more a developer has to do, the more willing they are to let the AI decide. One participant said both halves in a single sentence: "As much as I love coding, there is simply too much work to do to want to keep doing it myself." Stated preferences, one company, small effects; but the direction is what matters for the review queue. People keep the sign-off because the work matters to them, and they give it up when they have too much to do.

10a No time to learn the thing that saves time

slide at 10:45 · this step ~0:55

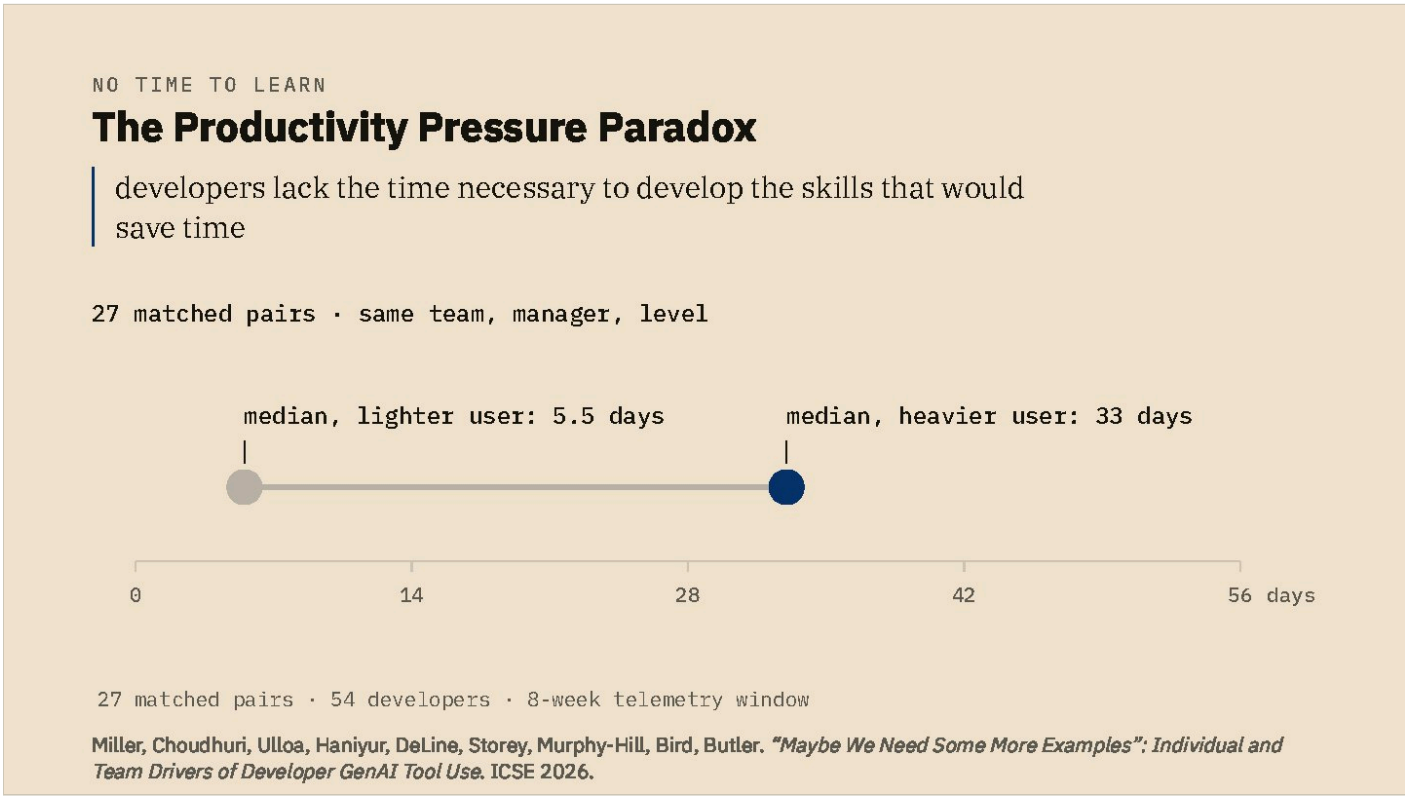


Miller, Choudhuri, Ulloa, Haniyur, DeLine, Storey, Murphy-Hill, Bird, Butler. *"Maybe We Need Some More Examples": Individual and Team Drivers of Developer GenAI Tool Use*. ICSE 2026.

And they do have too much to do. The people under this pressure have no time to get good at the tools. We found 27 pairs of developers on the same team, with the same manager and the same level, where one used Copilot about five and a half days in an eight-week window and the other thirty-three. We interviewed all fifty-four. Some of you saw the individual factors from this study last November; the paired design, the paradox and the systems argument are new. Both people in a pair hit the same failures. The frequent users treated it as a collaborator, experimented, and pushed through; the infrequent users tried it cautiously and took the first failure as the verdict, which under a deadline is the rational choice.

10b No time to learn the thing that saves time

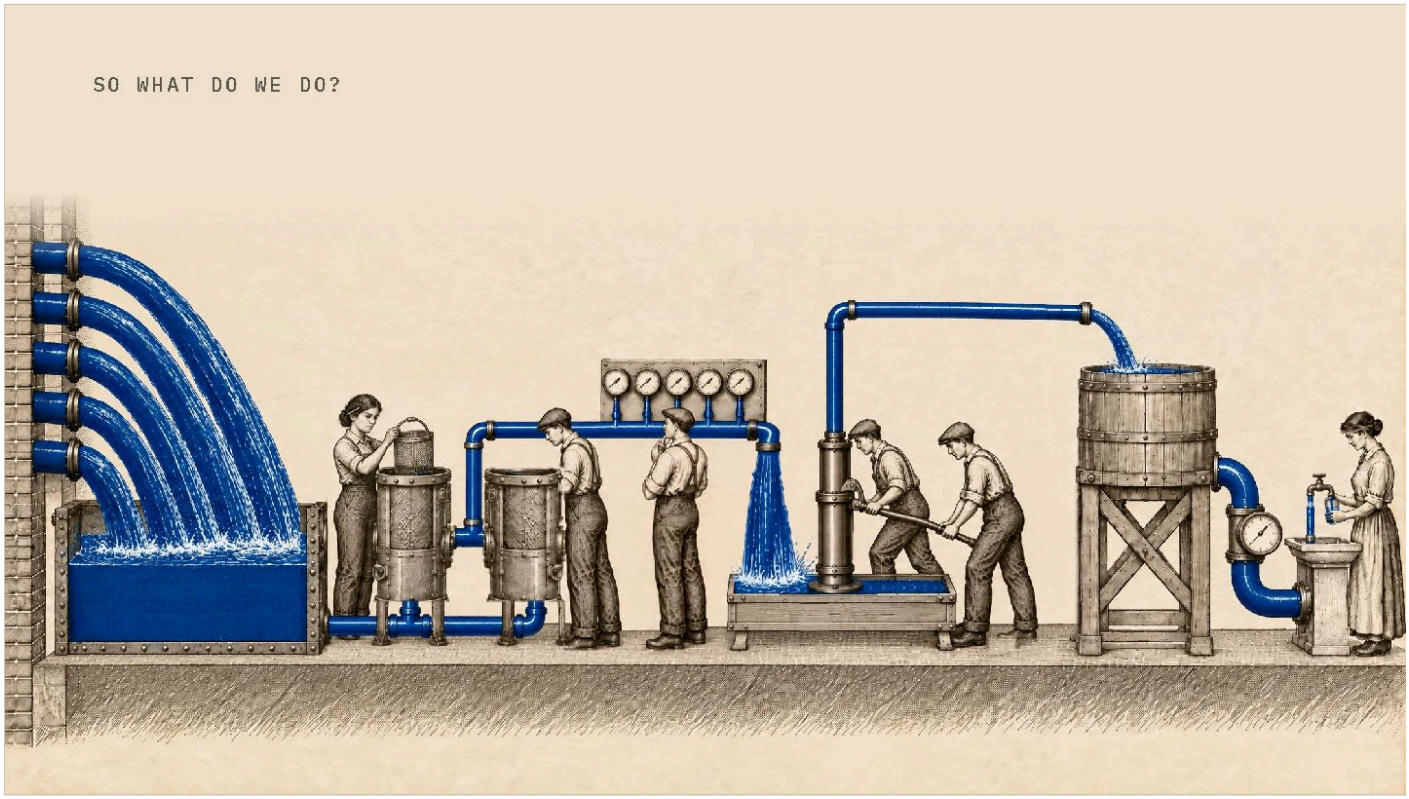
slide at 10:45 · this step ~0:24



The paper calls it the Productivity Pressure Paradox: "developers lack the time necessary to develop the skills that would save time." The obligation moves "away from individuals and back toward systems", with small things like a team expert who has ten percent of their time for it, or a recurring afternoon to try the tools together.

11a Rebuild for the new ratio

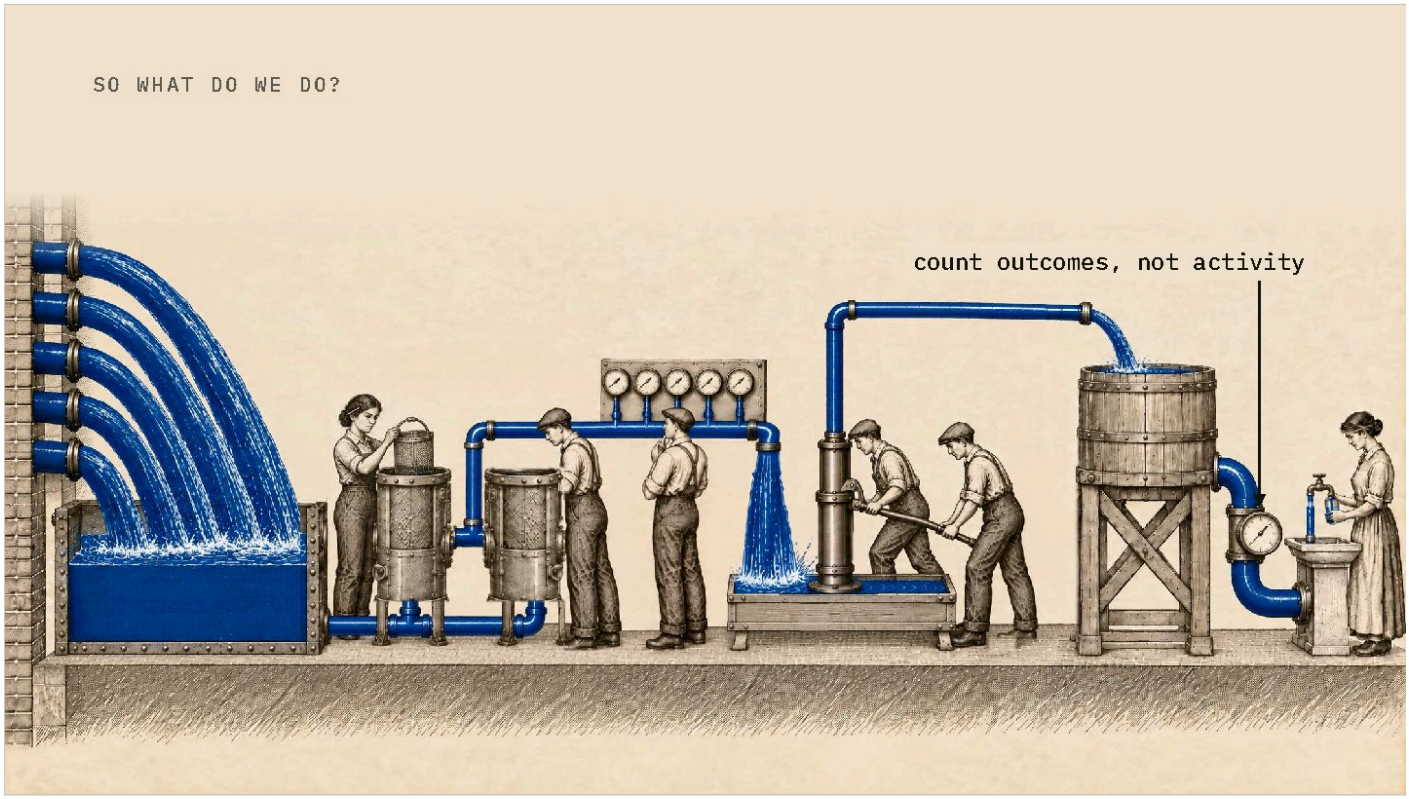
slide at 12:00 · this step ~0:07



So what do we do? Three things, and none of them waits for a better model.

11b Rebuild for the new ratio

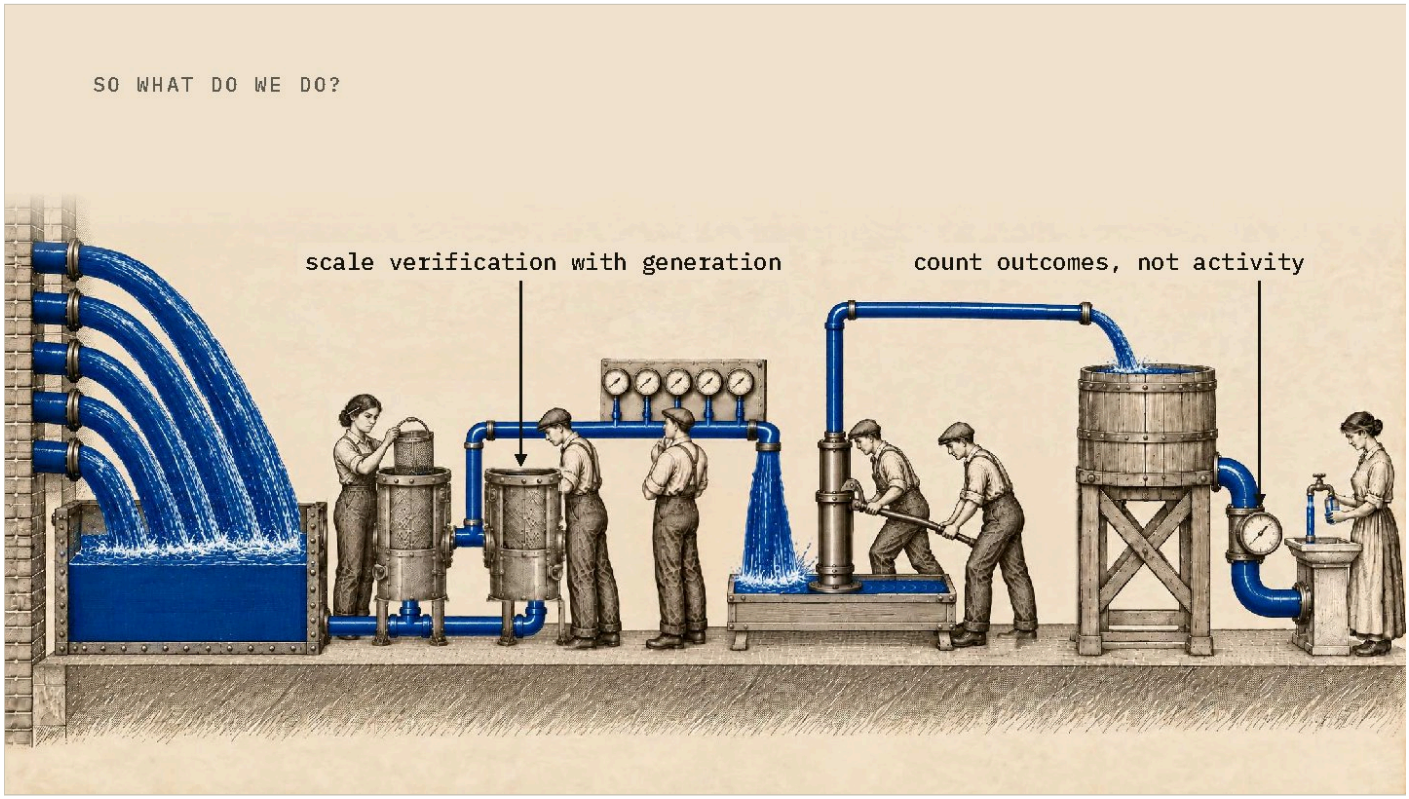
slide at 12:00 · this step ~0:07



Count outcomes, not activity. If your dashboard measures pull requests, it is measuring the water going in.

11c Rebuild for the new ratio

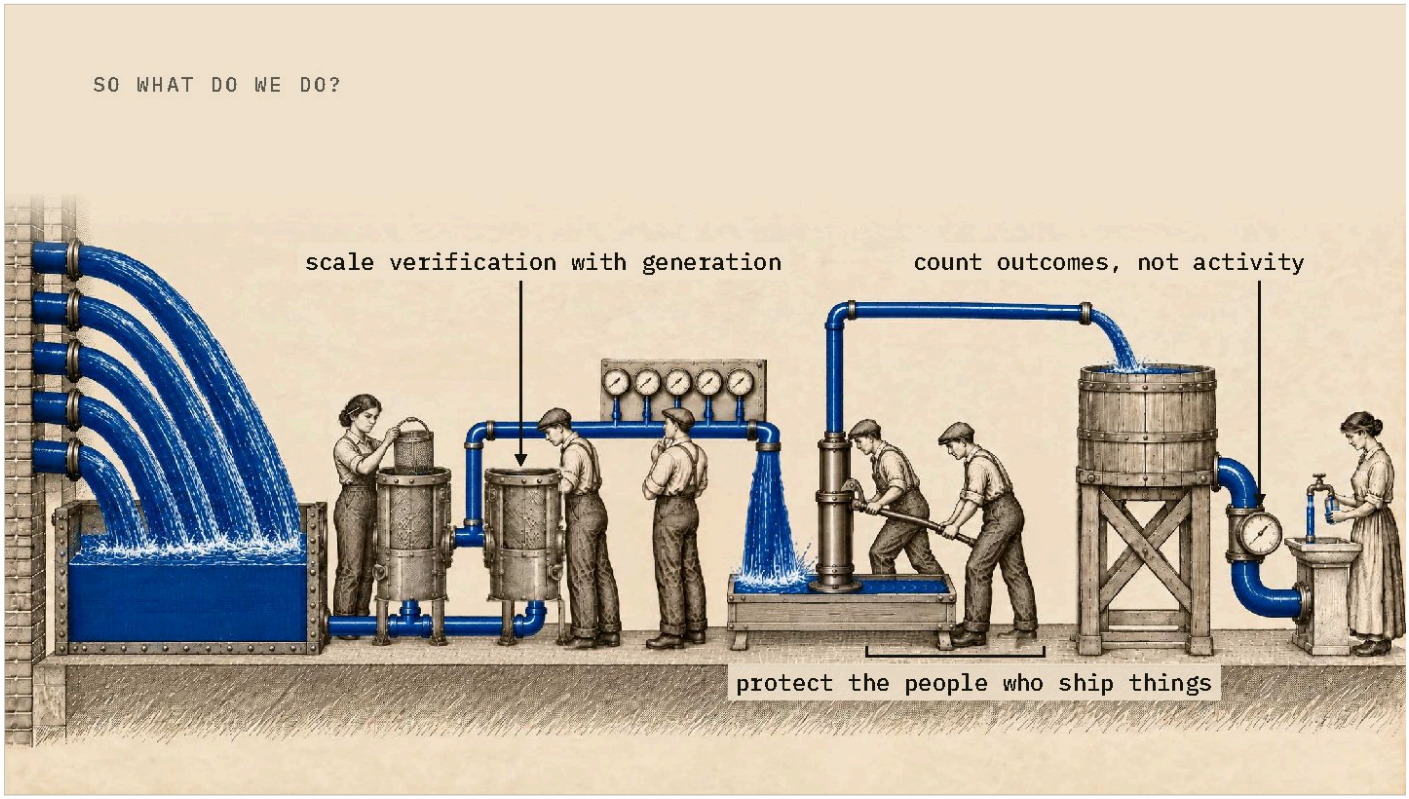
slide at 12:00 · this step ~0:07



Scale verification with generation: people, tools and time for review, in proportion to what you now generate.

11d Rebuild for the new ratio

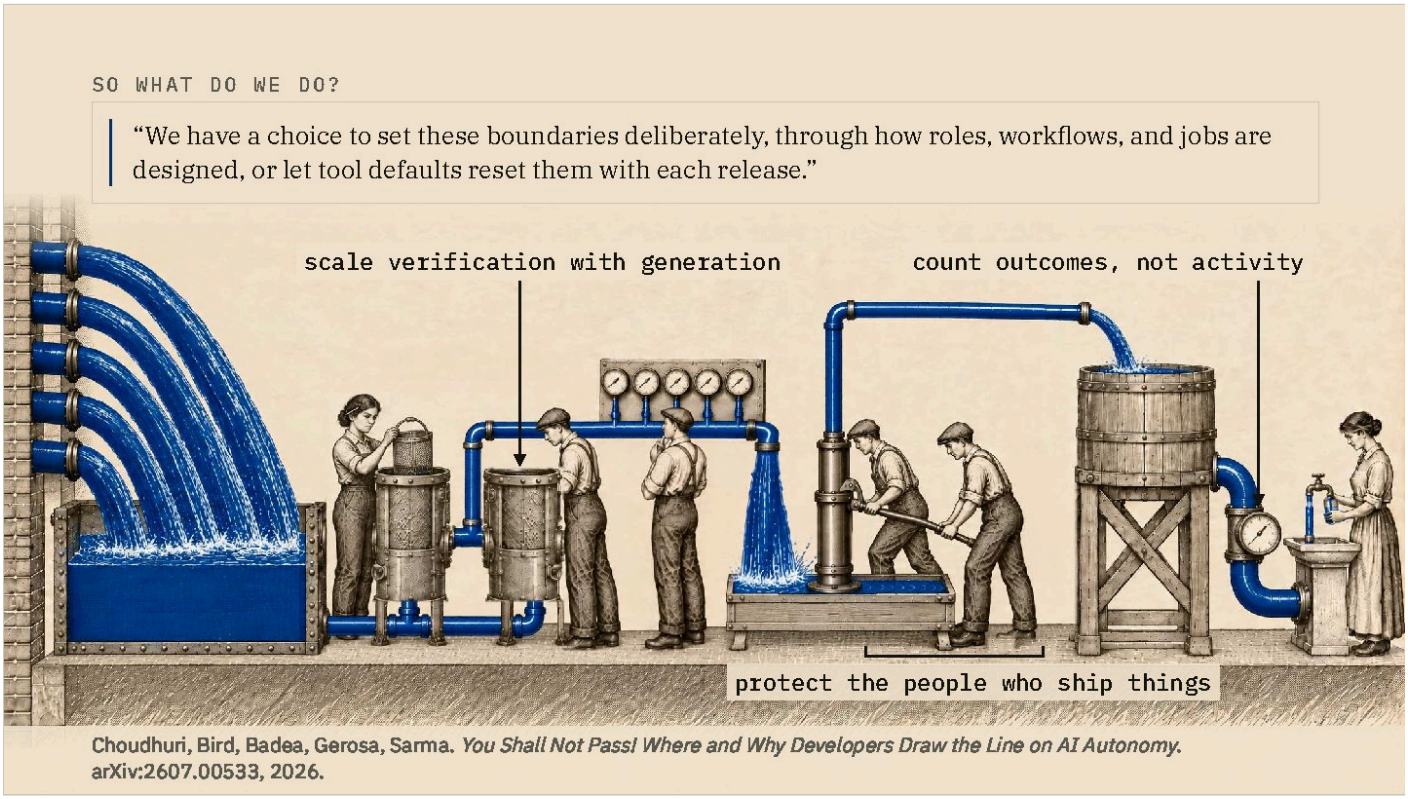
slide at 12:00 · this step ~0:22



And protect the people who ship things, and the time they need to learn and to teach, because that is the capacity the whole system is short of. Yoonha's paper adds a warning: as AI starts to speed up shipping too, organizations may respond by starting more work and tightening schedules.

11e Rebuild for the new ratio

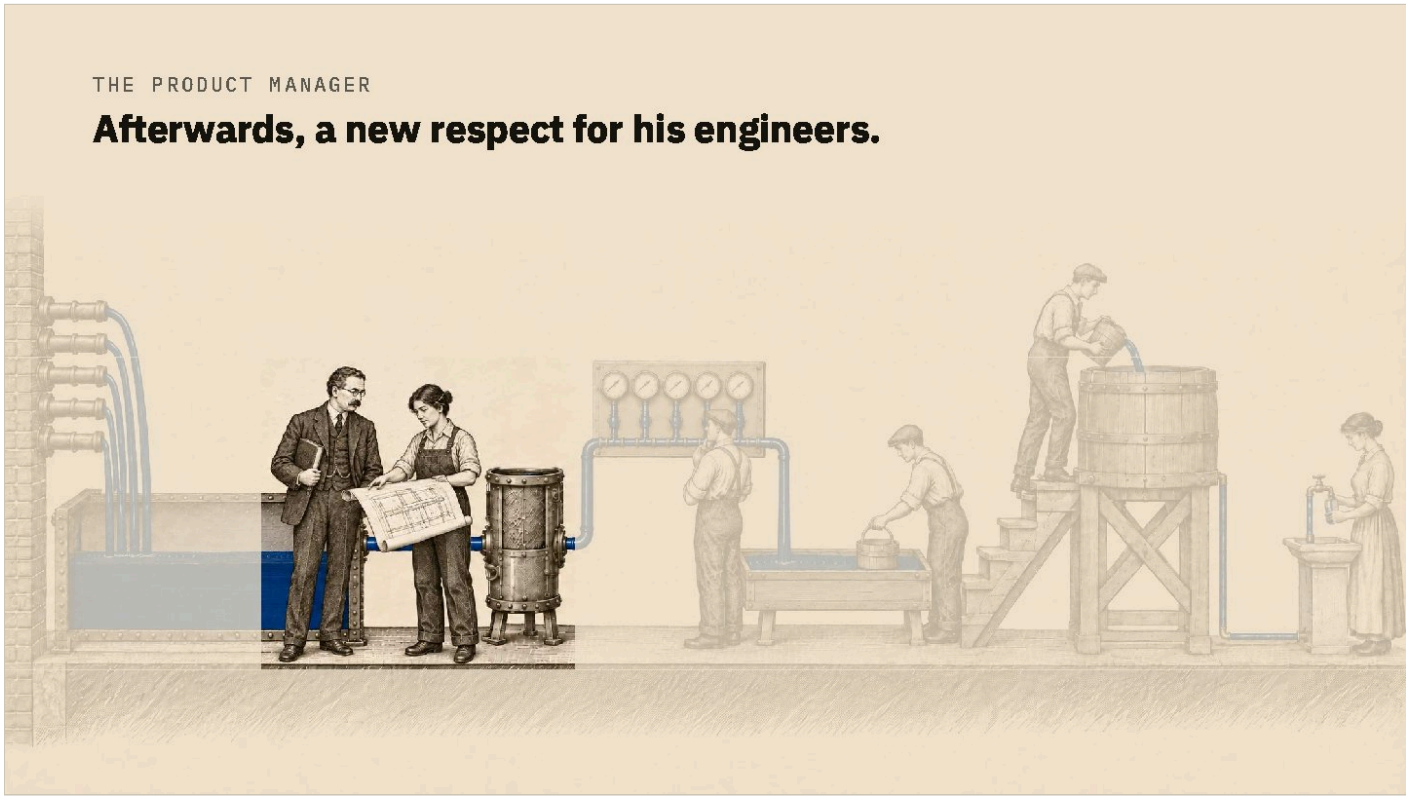
slide at 12:00 · this step ~0:15



The autonomy paper puts the choice this way: "We have a choice to set these boundaries deliberately, through how roles, workflows, and jobs are designed, or let tool defaults reset them with each release."

12a The answer

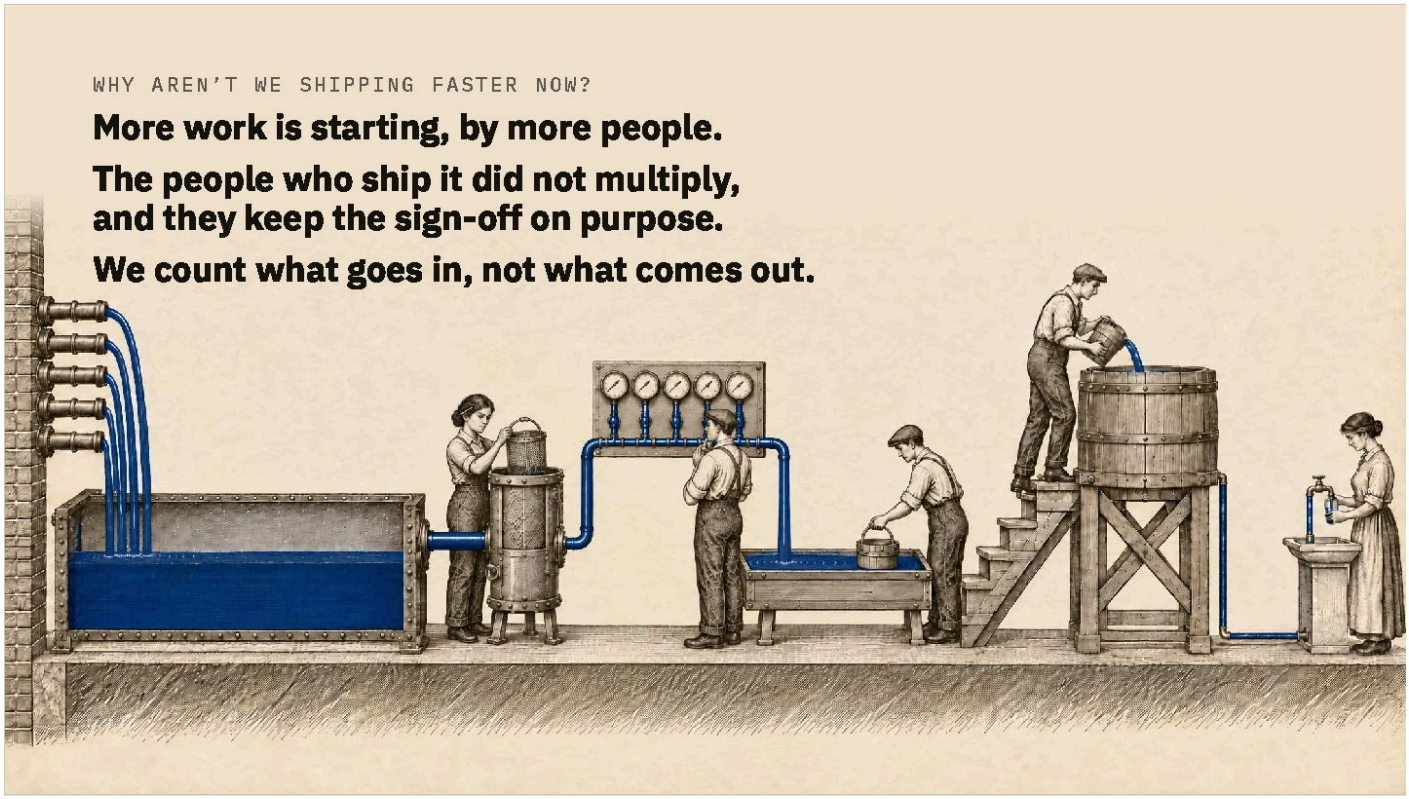
slide at 13:00 · this step ~0:19



The product manager who broke production? Afterwards, "he gained a new respect for his engineers." And just before he told me that story, my coauthor said something I keep coming back to: "I don't think it has to be this way." I think he's right.

12b The answer

slide at 13:00 · this step ~0:24



So why aren't we shipping faster now? More work is starting, by more people. The people who ship it did not multiply, and they keep the sign-off on purpose. And we count what goes in, not what comes out. Building the pipes on purpose is the organization's job, and we can tell them how. Thank you.